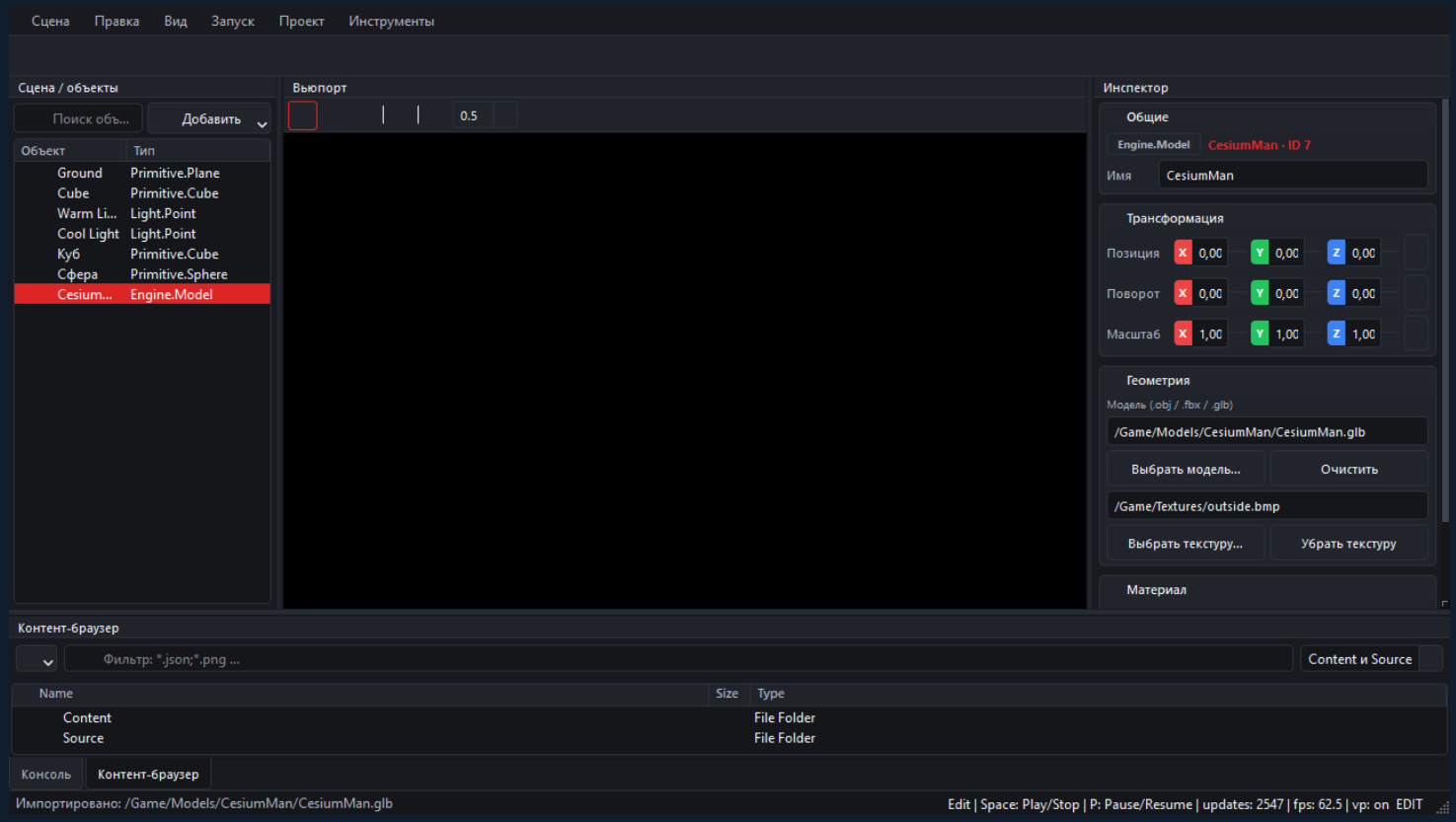
PainEngine Практический туториал

Редактор, материалы, звук и рабочие примеры C++



Снимки показывают Qt-панели; D3D viewport не входит в захват.

2026.11.07.6 / 07.11.2026

Первый проект в лаунчере

Создать проект

Выберите папку и название. C++ можно включить сразу или использовать шаблон без кода.

Редактировать

Выберите созданный проект в списке лаунчера и откройте редактор.

Флажок C++ создаёт исходники, включая RotatingCube. Нужны Visual Studio C++ tools и Windows SDK.

Окно редактора

Слева

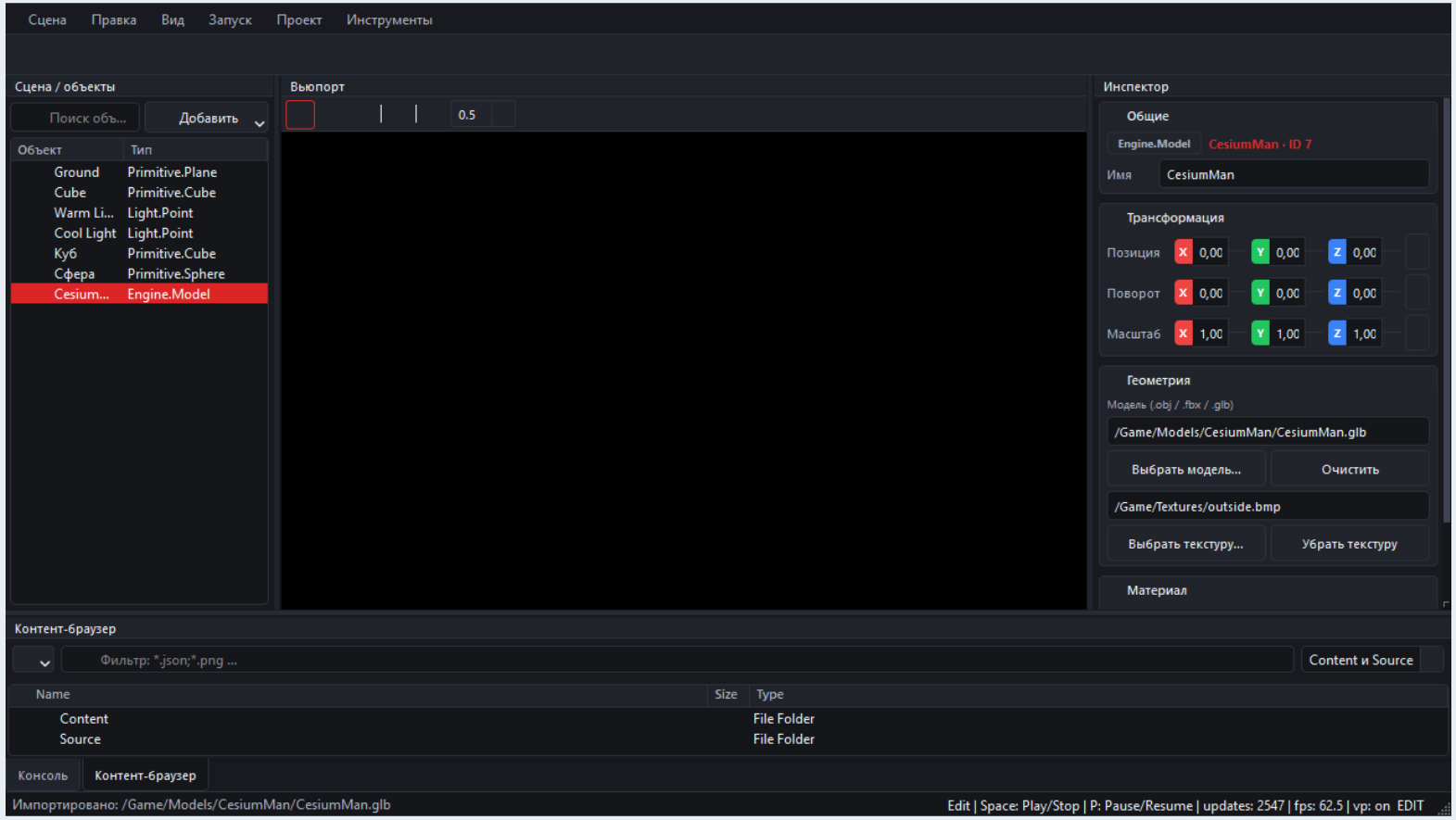
Каталог типов и объекты сцены. Выберите куб в дереве.

Справа

Сворачиваемые разделы и поля выбранного объекта.

Снизу

Контент-браузер; рядом вкладка Консоль. Высоту можно менять.



Панель контента: высота и положение

Изменить высоту

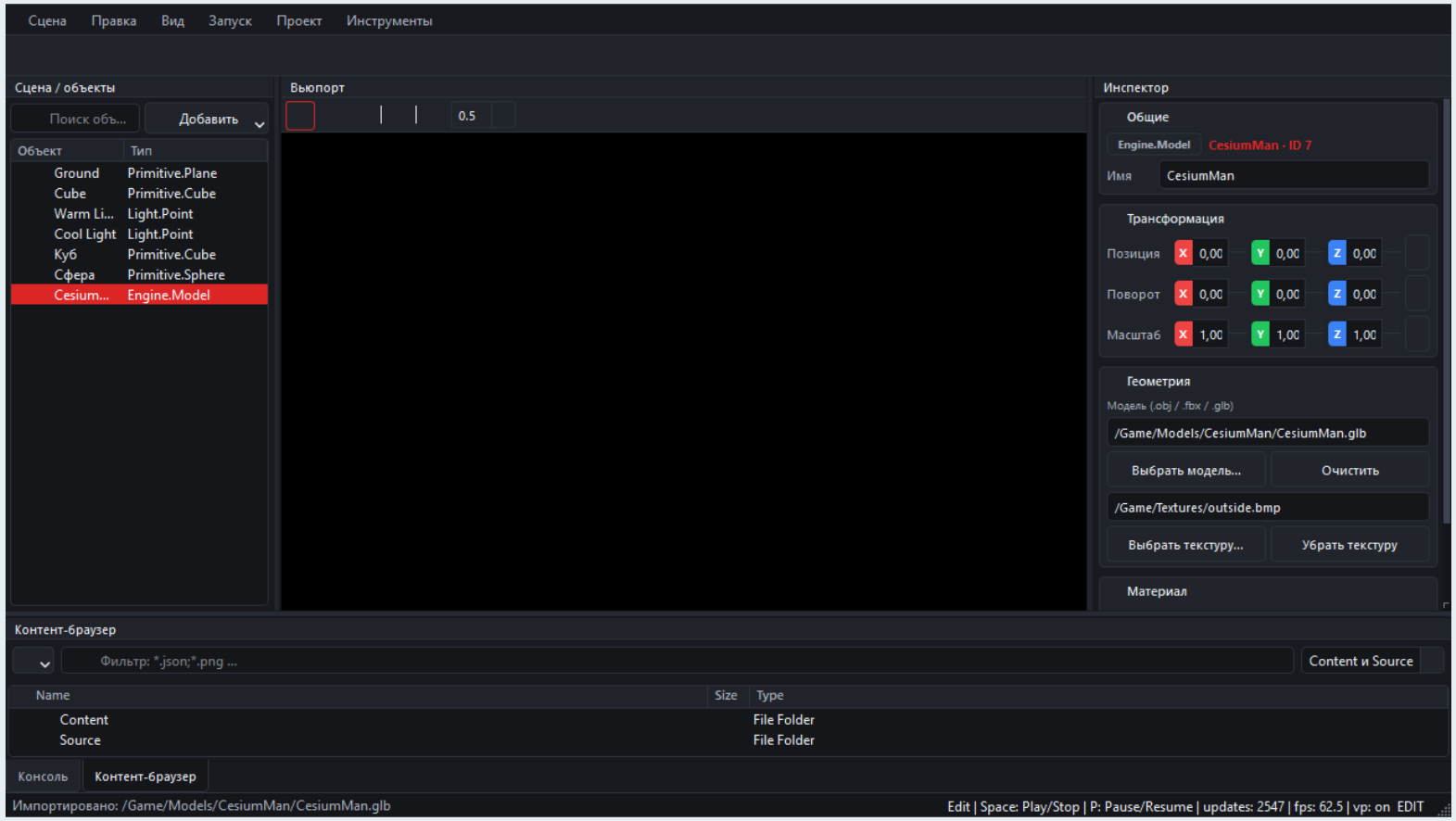
Тяните широкую серую границу над «Контент-браузер».

Переместить

Тяните заголовок или нижнюю вкладку панели.

Запомнить

Раскладка сохраняется отдельно для каждого проекта.



Граница меняет размер; заголовок и вкладка меняют место панели.

Вернуть удобную раскладку

Размер панели	Тяните разделитель между панелями.
Перемещение	Перетаскивайте заголовок панели в новое место.
Сброс	Вид → Сбросить раскладку окон.

Закрытую панель включите снова в меню «Вид».

Камера и выделение в Edit

Схема управления

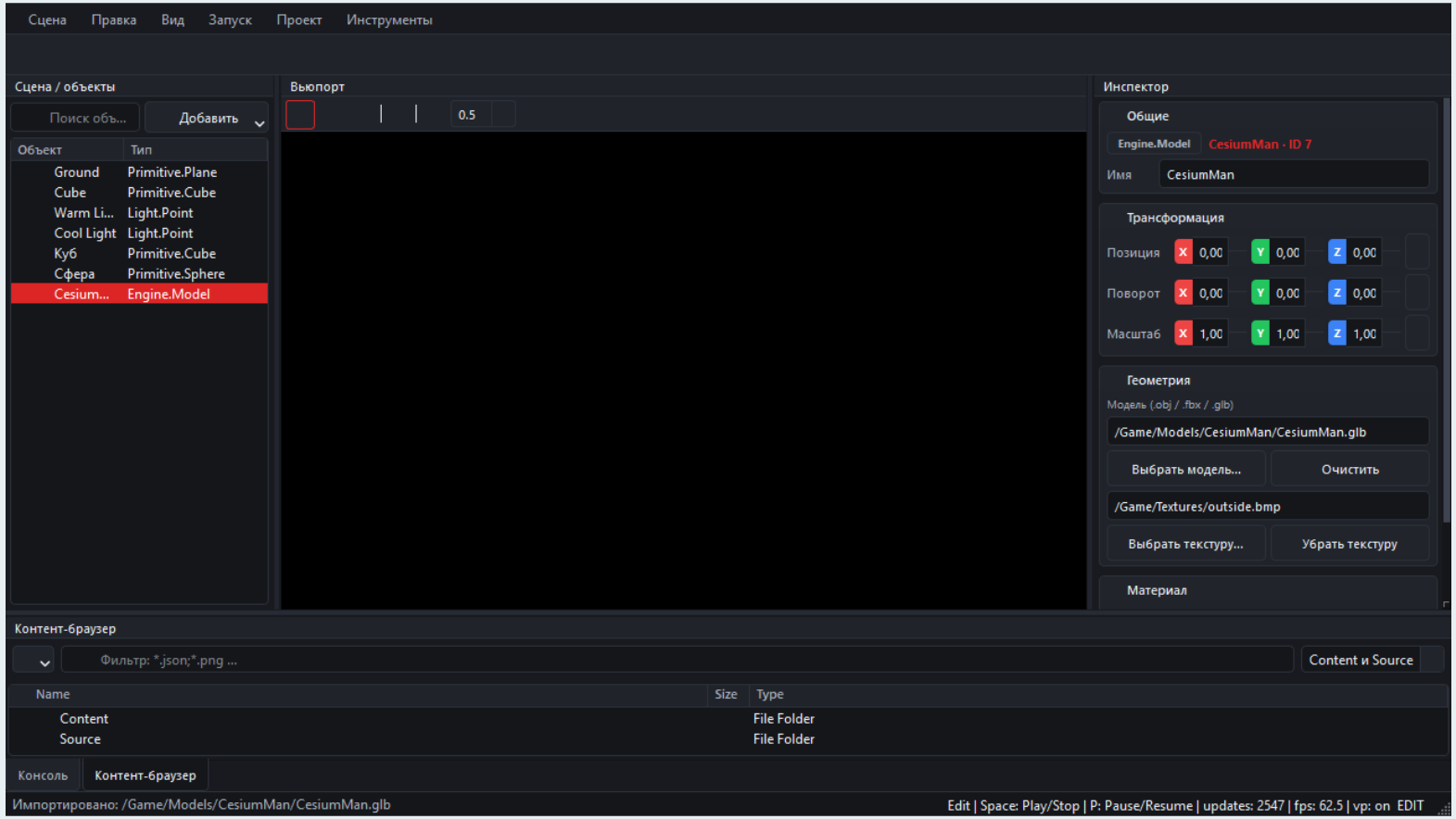
Правка → Схема управления: Unreal / Unity или Blender.

Выделение

Выберите объект в дереве или viewport. F фокусирует его.

Трансформация

Используйте gizmo и числовые поля инспектора.



Сначала щёлкните viewport. При вводе в текстовое поле горячие клавиши сцены не действуют.

Несколько объектов и привязки

Выбрать группу

Ctrl / Shift + щелчок в дереве или viewport. Трансформации применяются ко всей группе.

Переместить точно

Задайте шаг перемещения и угол на верхней панели. Ноль отключает привязку.

Отменить целиком

Ctrl+D, удаление и перетаскивание работают как одна операция Undo.

Остальные свойства относятся к активному объекту. Нужна актуальная DLL проекта.

Импорт модели и текстуры

Импорт

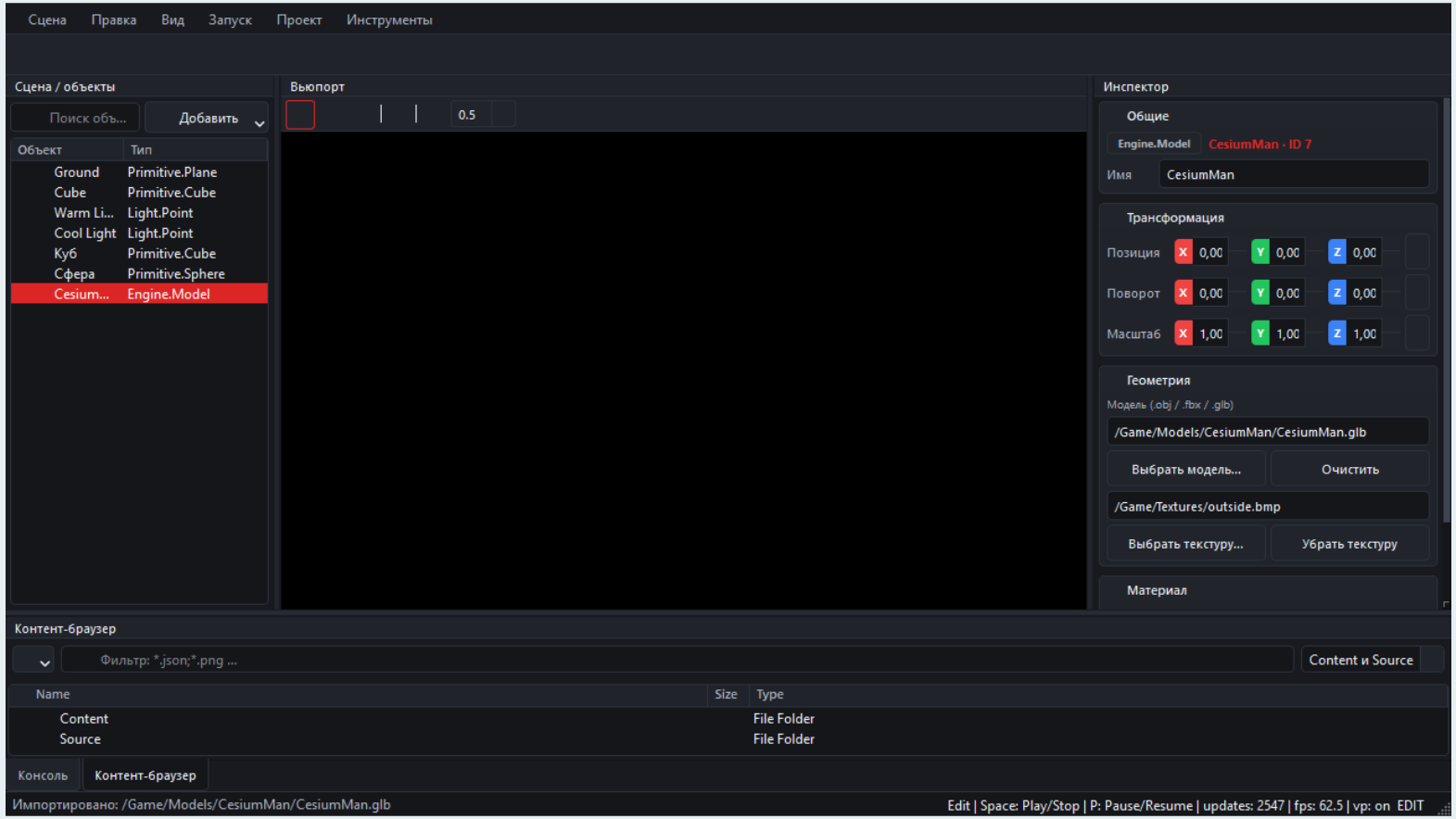
Сцена → Ресурсы → Импорт модели… или Импорт текстуры…

Добавление

Дважды щёлкните модель в Content/Models, чтобы добавить её в сцену.

Назначение

Для отдельной текстуры: выберите объект и нажмите «Выбрать текстуру…».



Модели: GLB, glTF, OBJ, FBX, BLEND. Изображения: PNG, JPG, BMP, TGA.

FBX / BLEND: модель сразу с материалом

Что создаётся

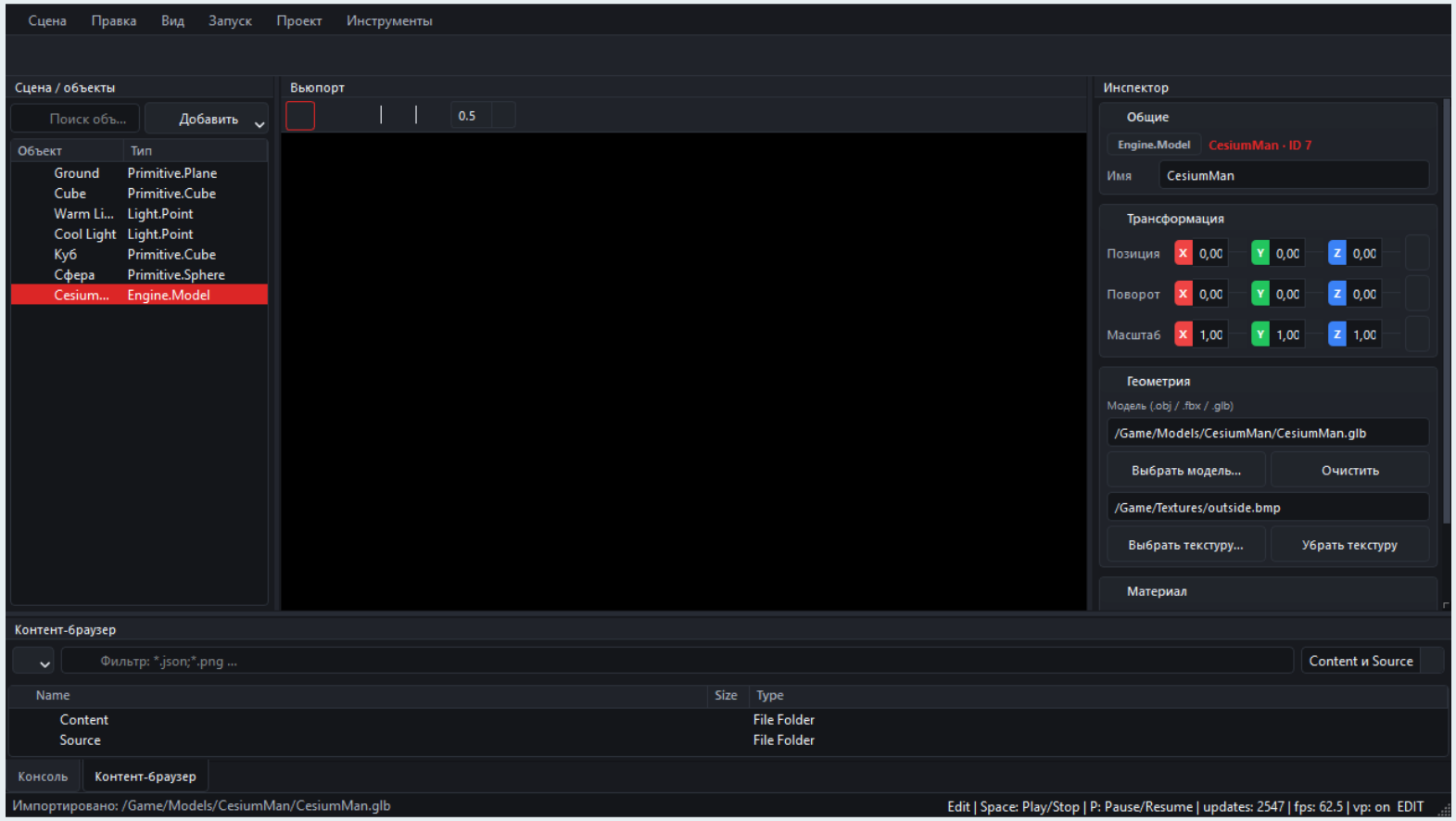
Модель, папки Materials и Textures, файл .import.json.

Материалы частей

Каждый source material ID получает .mat. Общий override заменяет все слоты.

Blender

Blender 4.2+. Переносится Image Texture из Base Color.



Базовый opaque-материал и UV0. Normal, roughness, metallic и сложный граф узлов не переносятся.

Обновить ресурс, сохранив сцену

Переимпортировать	ПКМ по модели или текстуре → Переимпортировать… Укажите исходный файл.
Перенести	ПКМ → Переместить / переименовать… Задайте новый путь внутри Content.
Проверить и сохранить	Посмотрите сцену и сохраните её. История Undo очищается; файлы остаются в резервной копии.

Модель переносится всей папкой; имя главного файла сохраняется. Операции доступны в Edit.

Редактор материала

Открыть

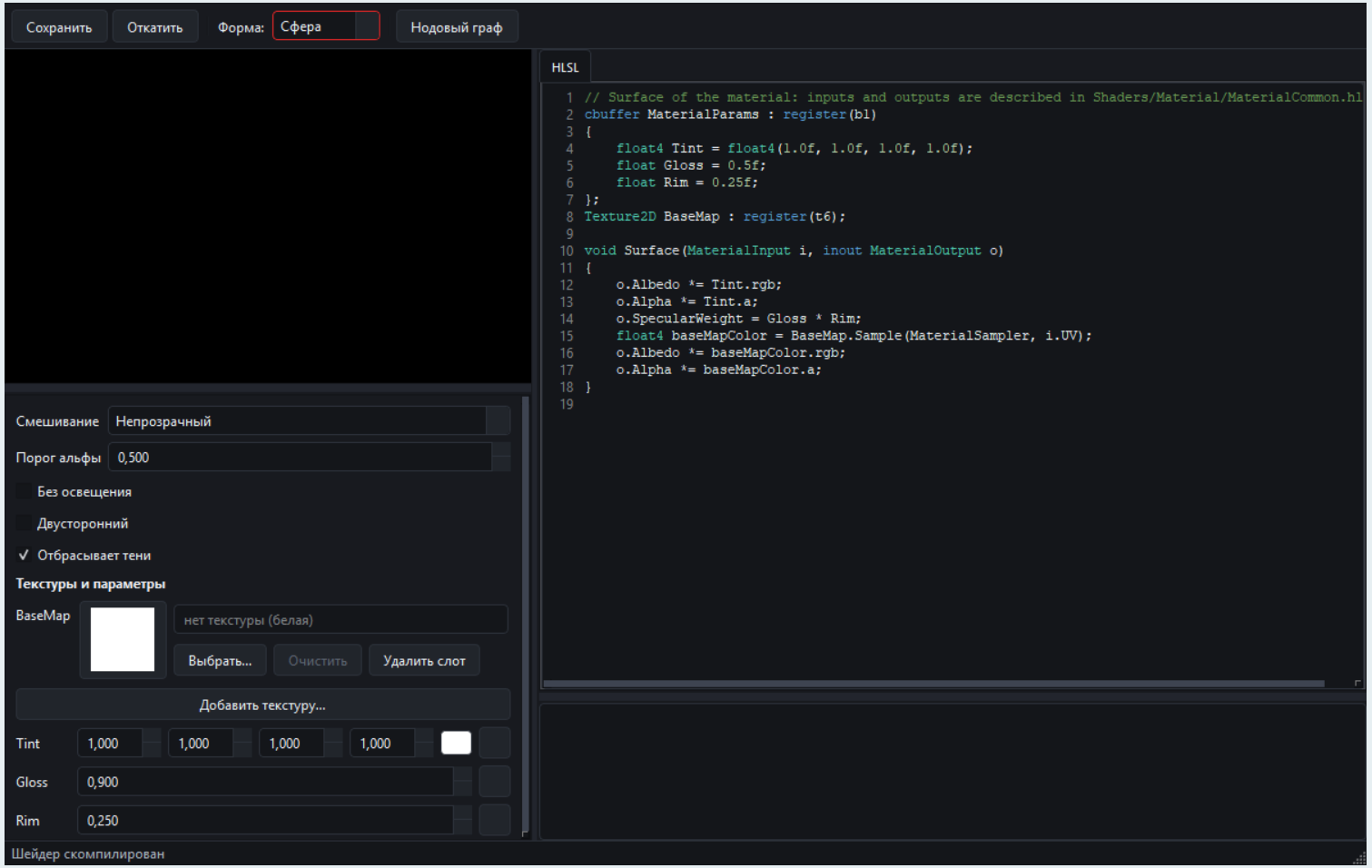
Свойства → Материал → Редактировать или двойной щелчок по .mat.

Режим материала

HLSL редактируется вручную или создаётся узловым графом.

Сохранить

Параметры видны сразу. Ctrl+S в этом окне сохраняет .mat и применяет HLSL.



Ctrl+S основного окна сохраняет сцену. F6 перечитывает материалы после внешних изменений.

Направленный свет и тени

Добавление

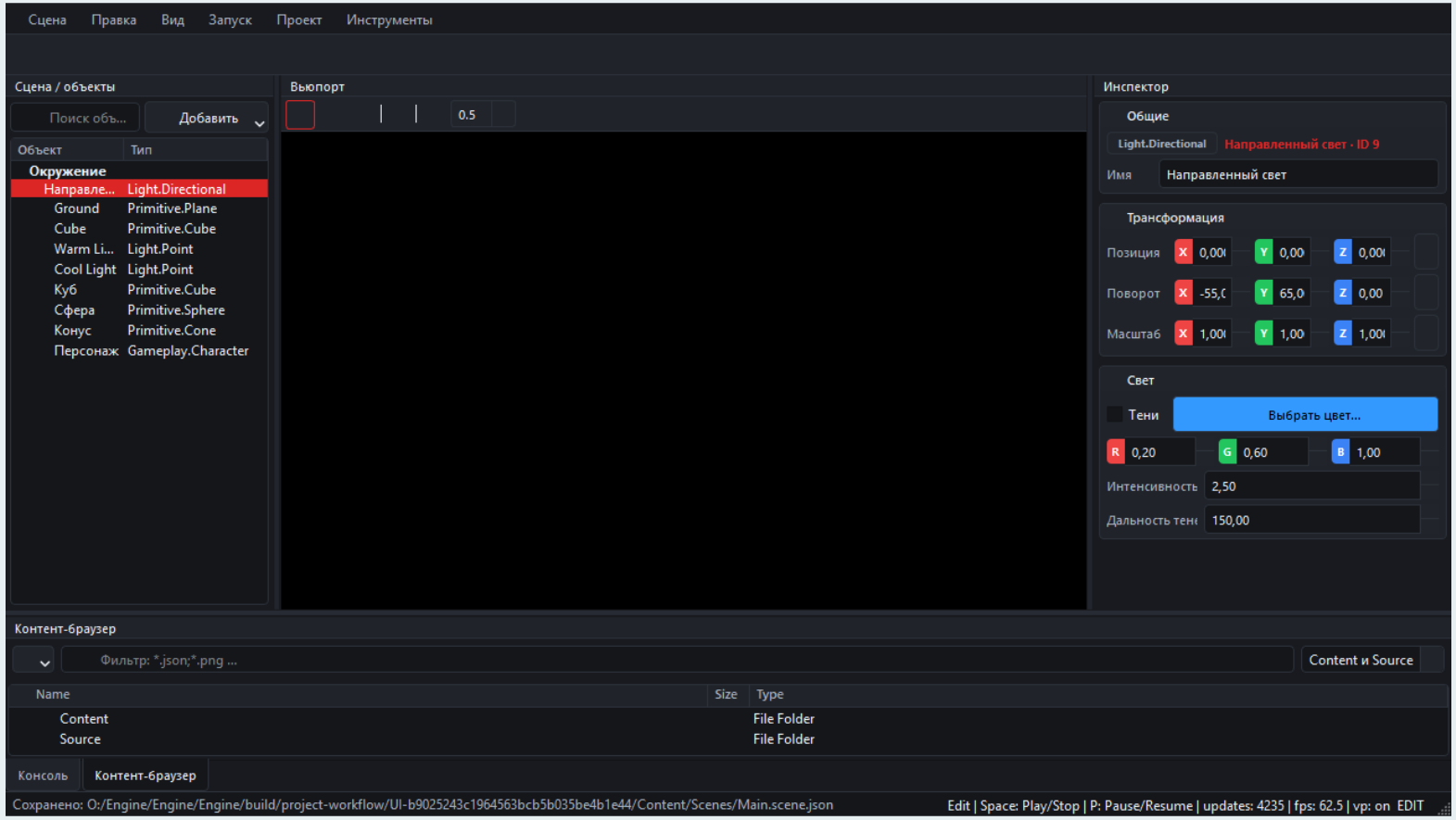
Выберите направленный свет в каталоге объектов сцены.

Повернуть

Поверните источник в трансформации. Позиция и масштаб не меняют свет.

Настроить

Задайте цвет, интенсивность, флажок и дальность теней.



Снимок показывает реальные панели. D3D-изображение viewport в этот захват не попало.

Точечный свет, частицы и небо

Точечный свет

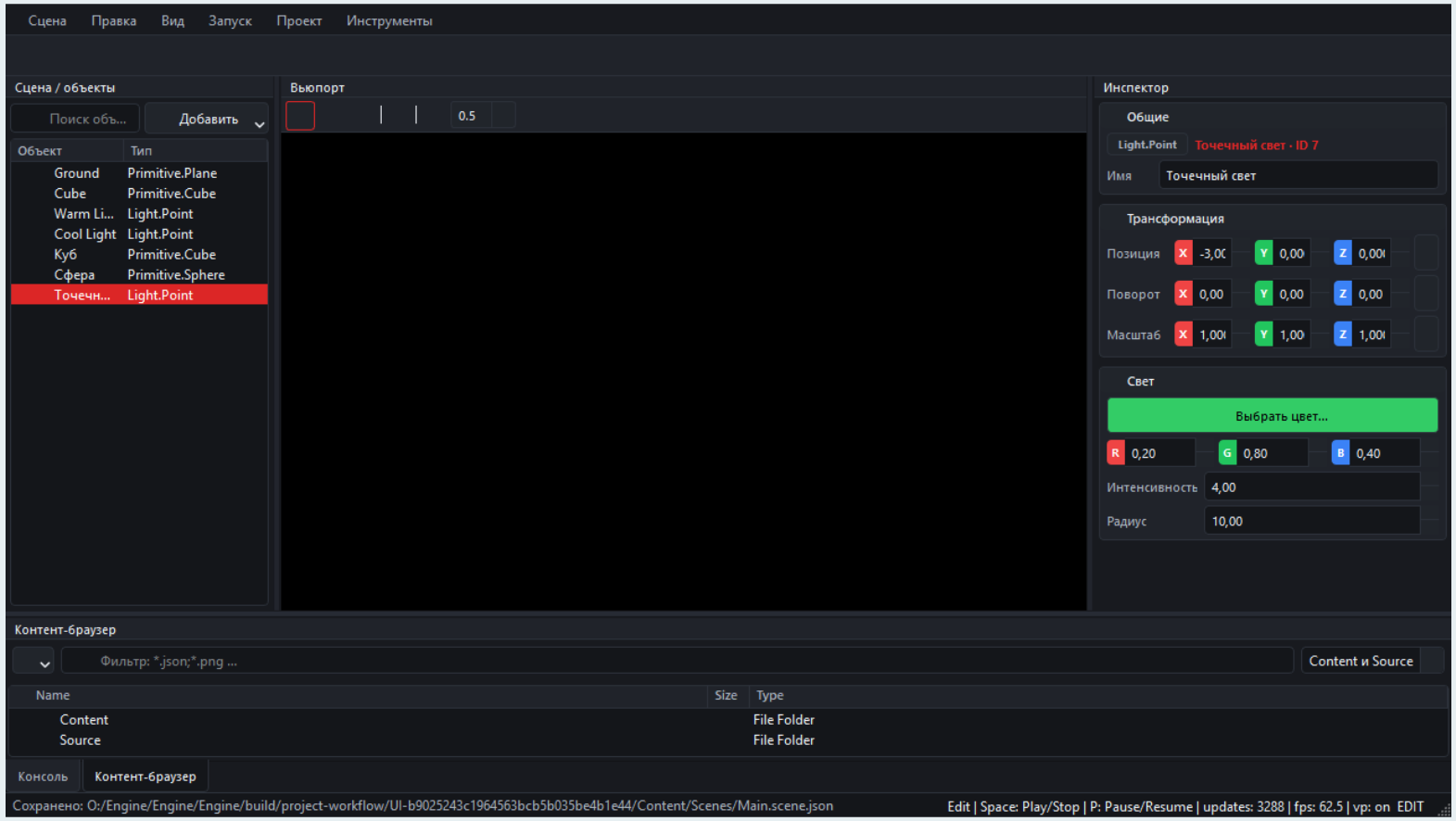
Выберите точечный источник. Настройте цвет, интенсивность и радиус.

Частицы

В категории «Частицы» выберите искры или дым.

Небо

Категория «Окружение» → «Базовое небо».



Параметры выбранного типа находятся ниже трансформации в правой панели.

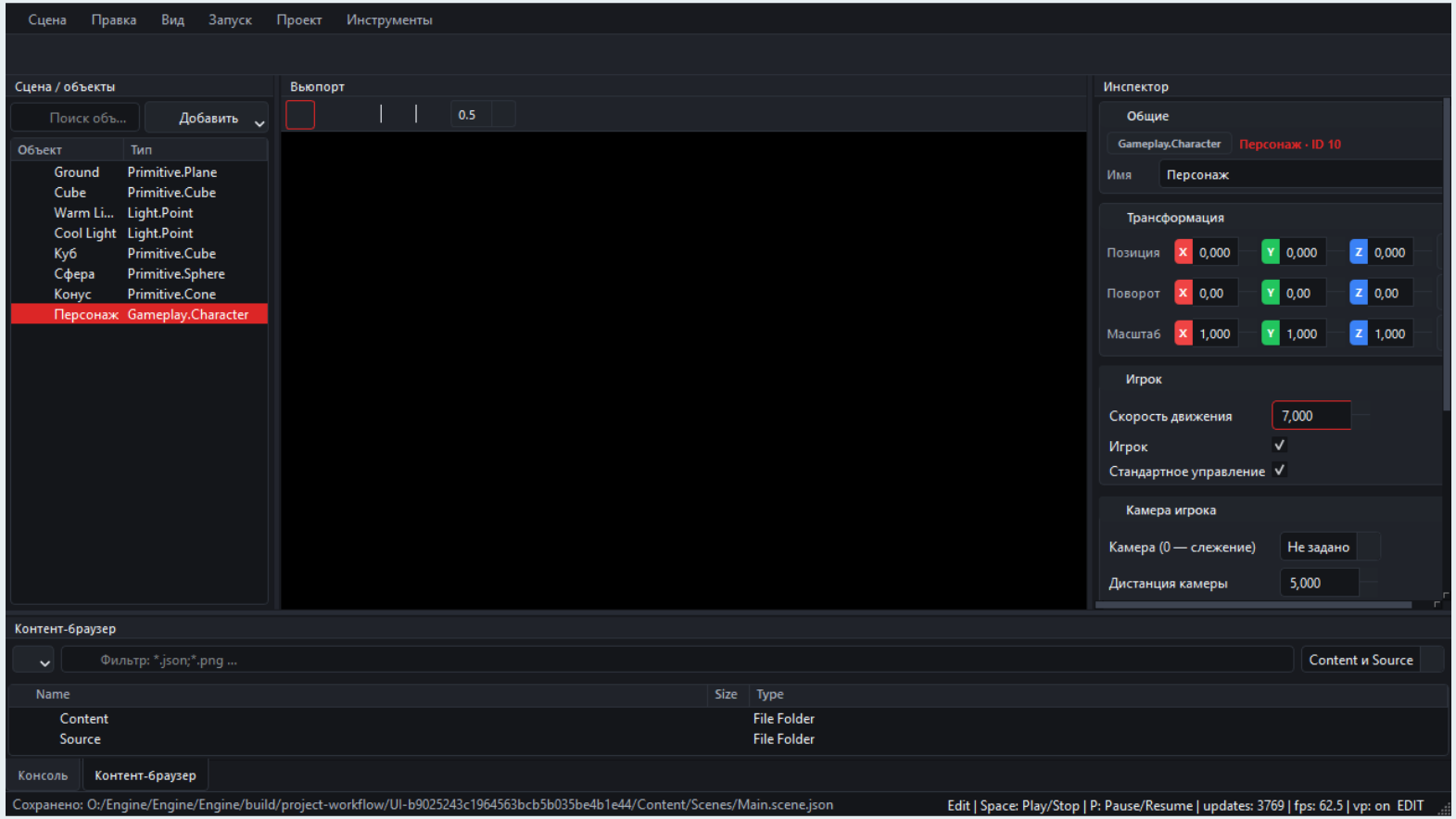
Стандартный персонаж

Добавление

Категория «Персонажи» → «Игрок» → «Добавить объект».

Управление

Выберите объект. Флажок «Игрок» и стандартное управление находятся в свойствах.



Игрок и камера: свойства объекта

Игрок

Включите флажок «Игрок» у персонажа. Для обычного управления включите «Стандартное управление».

Камера

Выберите слежение или отдельную камеру сцены. Дистанция, высота и чувствительность — в свойствах игрока.

Проекция и 2D

Задайте перспективу/FOV или ортографию/высоту вида, ближнюю и дальнюю границы, плоскость XY, XZ или YZ.

Без игрока остаётся камера редактора. Отдельного меню настройки Play больше нет.

Edit, Play, Pause и Stop

Play / Pause

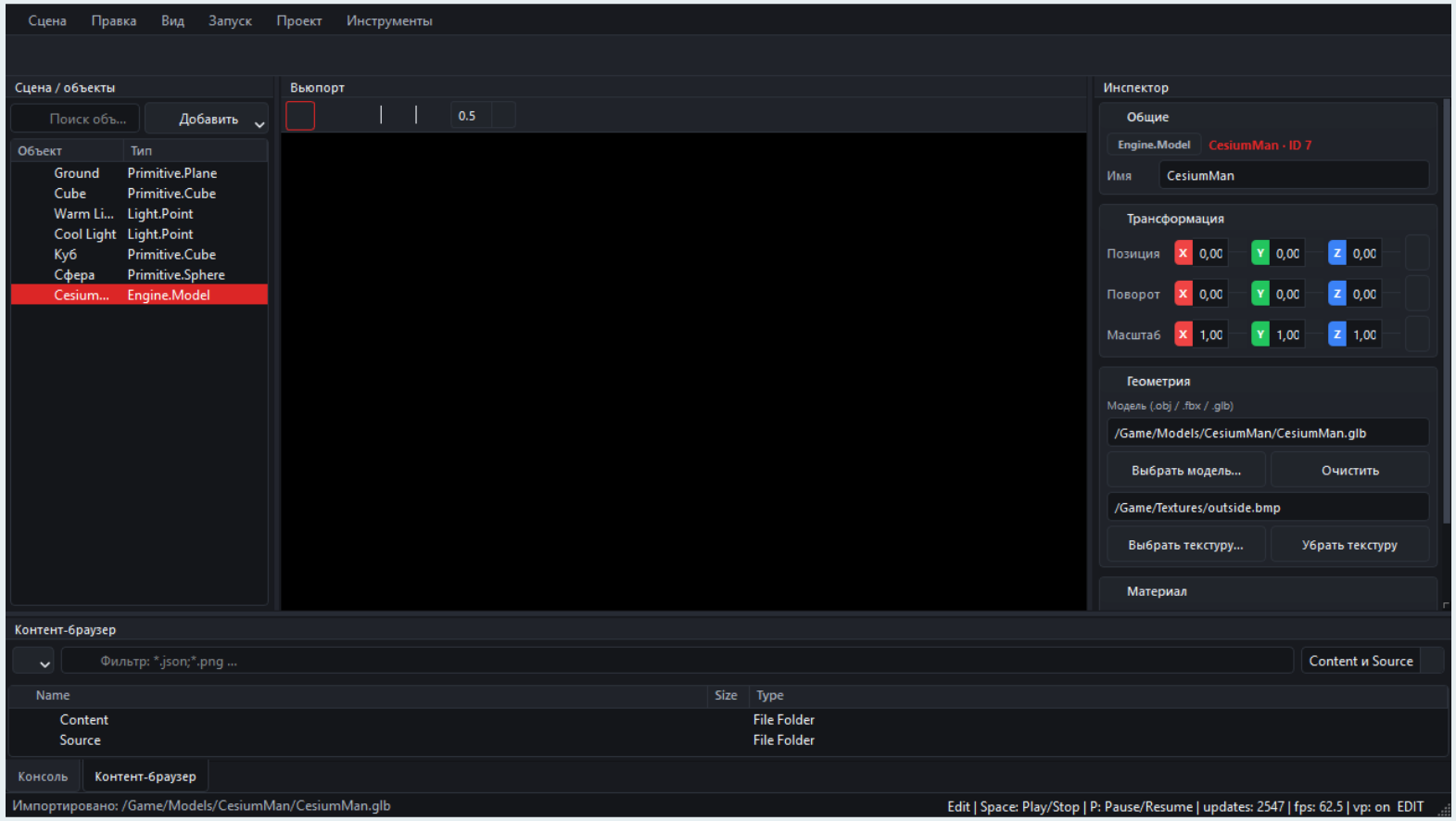
Play запускает копию сцены. Pause / Resume — пауза и продолжение.

Stop

Возвращает рабочую сцену и камеру Edit.

Отдельное окно

Отдельный запуск использует сохранённые файлы проекта.



В viewport: Space — Play / Stop, P — Pause / Resume.

Сохранение и структура проекта

Файл / папка	Назначение
Project.json	Пути к модулю, Content и стартовой сцене
Engine.lock.json	Закреплённая версия и конфигурация движка
Content/Scenes	Сцены с объектами, трансформациями и настройками Play
Content/Models, Textures, Materials	Модели, изображения, файлы .mat и .hlsl
Source и Build.ps1	Код и запуск сборки C++ проекта
Binaries	Собранные версии DLL проекта

Ctrl+S сохраняет сцену. Стартовая сцена назначается отдельно: Сцена → Сделать стартовой сценой.

Восстановление и передача игры

После сбоя

При открытии восстановите предложенную копию сцены. Затем сохраните её через «Сохранить как…».

Перед экспортом

Назначьте стартовую сцену и сохраните материалы. Выберите «Экспорт игры…».

Передать папку

Укажите новую папку вне проекта. Запуск у получателя — через Start.cmd.

Снимок сцены создаётся каждые 30 секунд. Экспорт доступен из Release-редактора.

Подключить HUD без C++

Настройки игры

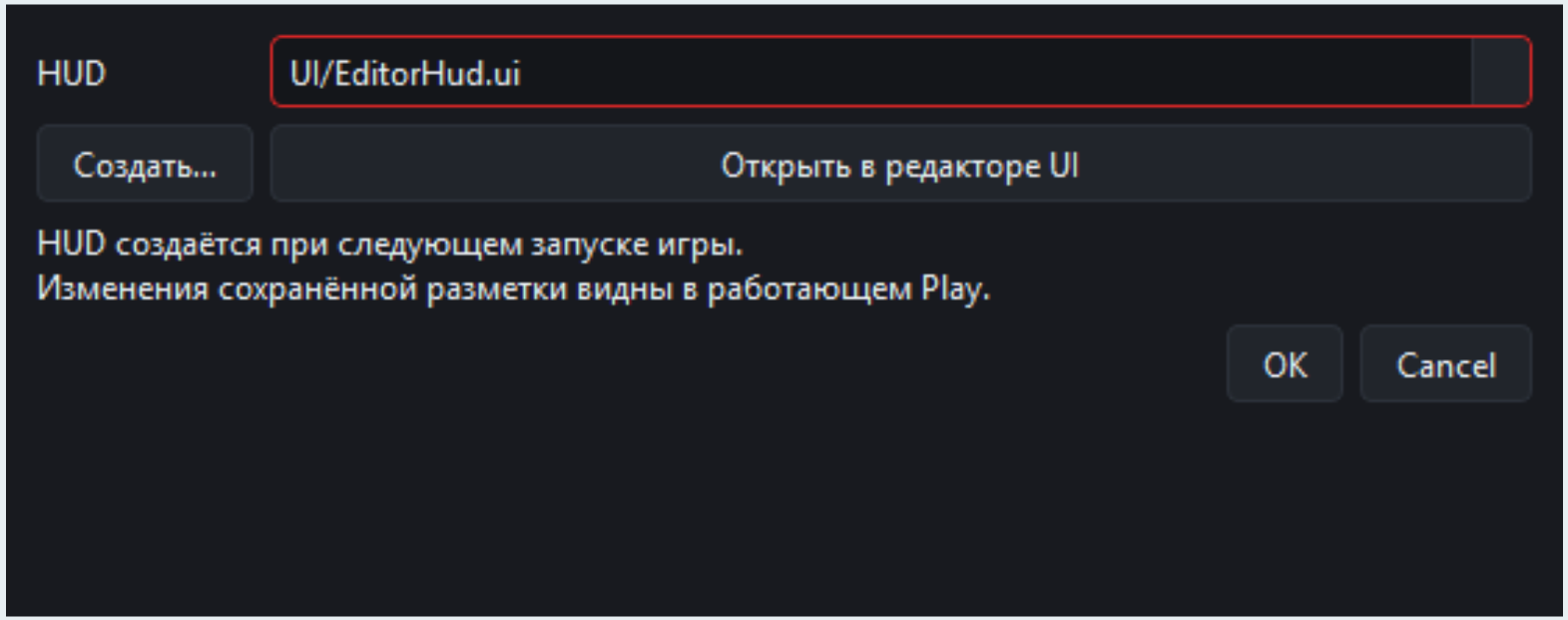
Откройте «Запуск → HUD игры…». Начальное значение — «Нет».

Выберите интерфейс

Выберите .ui или создайте новый. Подтвердите выбор кнопкой OK.

Запустите Play

HUD появится в игре. После Stop игровая копия удалится.



Выбор относится ко всему проекту и сохраняется между запусками редактора.

Минимальный редактор интерфейса

Откройте файл

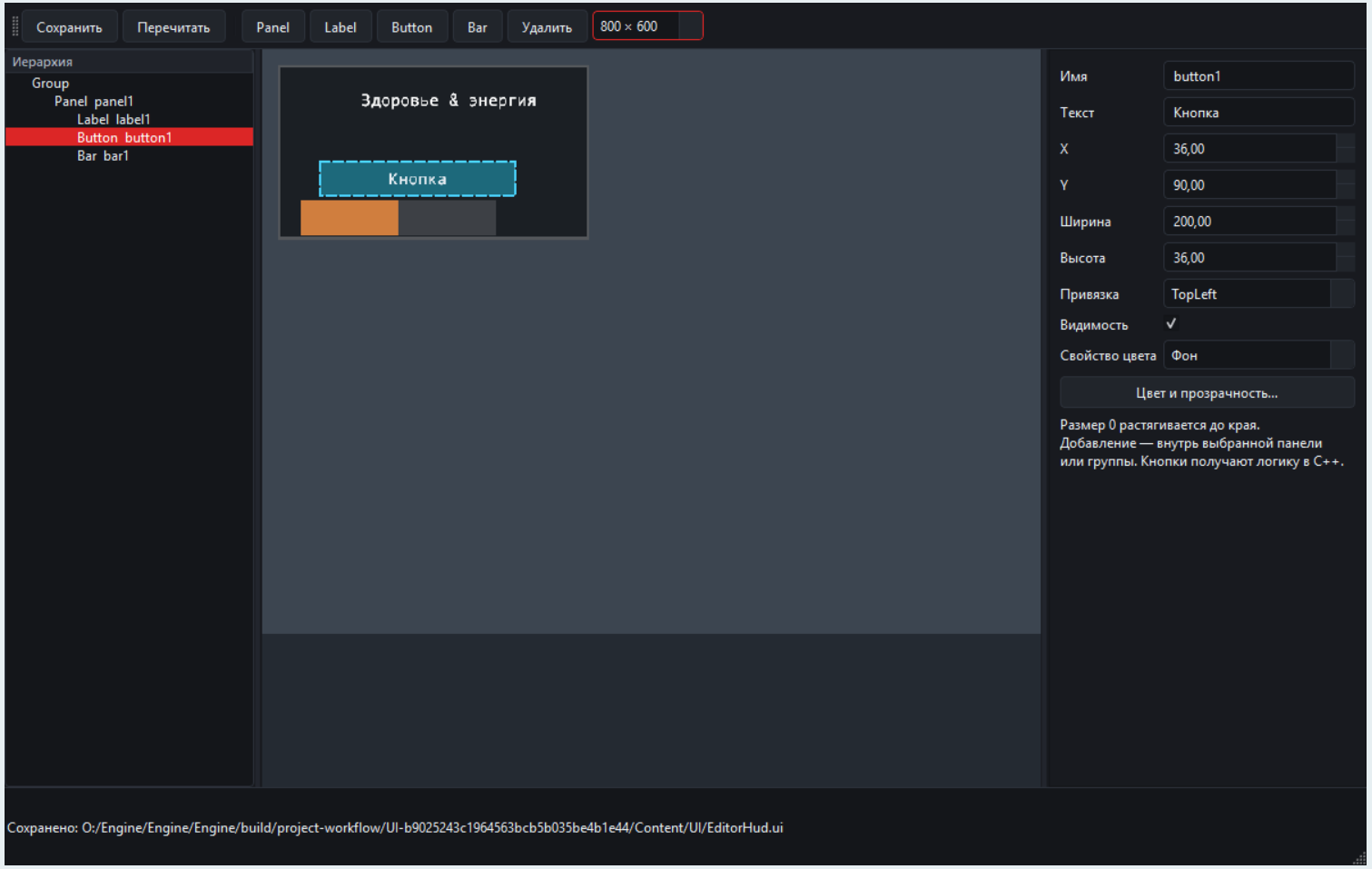
Откройте .ui двойным щелчком в Content или через меню UI.

Соберите иерархию

Добавьте Panel, Label, Button и Bar. Перемещайте элементы мышью.

Настройте вид

В инспекторе меняйте текст, размеры, привязку, цвет и прозрачность.



Холст работает без Play. Игра и предпросмотр используют одни правила вёрстки.

Проверка HUD в Play

Сохраните и запустите	Нажмите «Сохранить» в редакторе UI. Выберите этот HUD и запустите Play.
Измените внешний вид	Поменяйте цвет или положение, сохраните .ui и вернитесь в игровой viewport.
Проверьте отключение	Остановите игру, выберите «Нет» в настройках HUD и снова запустите Play.

Сохранение разметки не требует сборки C++. Действия кнопок задаются кодом игры.

Граница между редактором и C++

Задача	Без кода	Через SDK / C++
Сцена	Модели, свет, материалы	Свои компоненты и данные
Персонаж	Выбор для Play	Свой Character
Камера Play	Слежение, камера сцены	Свой UpdateCamera
Управление	WASD и ПКМ	Действия и обработчики
Свойства	Значения полей	Описание полей в C++
Игровой HUD	Выбор и редактор .ui	Данные и действия кнопок

Контроллер создаётся игрой и не является отдельным добавляемым объектом сцены.

Создание C++ класса

Мастер

Проект → C++ → Добавить C++ класс… Выберите родителя и задайте имя.

Исходники

Мастер создаёт Source/Имя.h и Source/Имя.cpp.

Сборка

Ctrl+Alt+R — сборка и Hot Reload. Выполняйте в Edit.

Для имени класса: латиница, цифры и _. Начинайте с буквы или _.

Базовый C++ проект: вращающийся куб

Готовый пример

RotatingCube.h/.cpp создаются вместе с C++ проектом.

Запуск

ProjectGame добавляет куб при Play или отдельном запуске.

В Edit

Выберите «Классы проекта» → RotatingCube и добавьте объект.

По умолчанию 90°/с; новый шаблон даёт поля Speed и Enabled в Behavior → Rotation.

RotatingCube: минимальный класс

```
// Source/RotatingCube.h
#pragma once
#include <Engine/Components/ComponentAnnotations.h>
#include <Engine/Components/SpecificComponents/CubeComponent.h>

ENGINE_COMPONENT("RotatingCube")
class RotatingCube : public CubeComponent {
public:
    RotatingCube();
    void Tick(float deltaTime) override;
};

// Source/RotatingCube.cpp
#include "RotatingCube.h"
RotatingCube::RotatingCube() {
    SetColor({1.0f, 0.65f, 0.2f, 1.0f});
}
```

ENGINE_COMPONENT добавляет класс в каталог проекта.

CubeComponent уже имеет геометрию куба.

Конструктор задаёт исходный вид.

Вращение на месте в Tick

```
#include "RotatingCube.h"
#include <cmath>

void RotatingCube::Tick(float deltaTime)
{
    CubeComponent::Tick(deltaTime);
    auto rotation = GetRotation();
    rotation.y = std::fmod(
        rotation.y + 90.0f * deltaTime, 360.0f);
    SetRotation(rotation);
}
```

SetRotation принимает градусы.

deltaTime — время шага в секундах.

Позицию не меняем, поэтому куб вращается на месте.

В ProjectGame: SetSimulationTimeScale(1.0f). Полный оборот — четыре секунды.

Практика: два куба, один класс

Учебные исходники	Скопируйте PropertyTutorial в новый C++ проект по README.
Найти поля	Выберите «Медленный куб…». Раскройте «Свойства туториала».
Изменить скорость	В разделе «Вращение»: 45 → 360, Enter. Play покажет результат.

Исходники и сцена: tutorial/examples/PropertyTutorial. У каждого куба свои значения.

Проверьте пользовательские свойства

Действие в Edit	Ожидаемый результат
Скорость: 45 → 360	В Play полный оборот за одну секунду
Скорость: отрицательное число	В следующем Play обратное направление
Снять «Вращение включено»	В Play выбранный куб стоит
Яркость куба: 1 → 0,2	Цвет меняется сразу, без Play
Отменить / Повторить	Возвращается прежнее / новое значение
Сохранить и открыть сцену	Сохраняются отдельные настройки обоих кубов

После проверки нажмите Stop. Для изменения уже объявленного поля пересборка не нужна.

Как поле появляется в инспекторе

```
#include <Engine/Components/PropertyMacros.h>
#include <Engine/Components/SpecificComponents/CubeComponent.h>

ENGINE_COMPONENT("ExampleCube")
class ExampleCube : public CubeComponent {
public:
    ENGINE_GENERATED_PROPERTIES(CubeComponent)
private:
    ENGINE_PROPERTY("Скорость, °/с", -720, 720)
    float speed = 90.0f;

    ENGINE_PROPERTY("Вращение включено")
    bool enabled = true;

    ENGINE_PROPERTY("Яркость", 0.1, 1.0)
    float brightness = 1.0f;
};
```

Аннотация стоит над полем.

Генератор создаёт описание и сохранение при сборке.

Дополнительное наследование не требуется.

Имя поля — ключ сохранённых данных. Поля без аннотаций остаются обычным состоянием C++.

Своё поле: данные → вид → поведение

Объявить

Добавьте поле и ENGINE_PROPERTY над ним. Диапазон и подпись задаются в аннотации.

Собрать

ENGINE_GENERATED_PROPERTIES(Base) объявляет методы. ComponentGenerator реализует их при сборке DLL проекта.

Применить

В Tick используйте значение поля. Для немедленного обновления внешнего вида добавьте public OnPropertiesChanged().

Для новых полей старых сцен остаётся значение конструктора. Переименование ключа требует миграции.

Создание объекта в мире

```
// В ProjectGame::AfterInitialize()
Game::AfterInitialize();
SetSimulationTimeScale(1.0f);

auto& scene = GetScene();
std::unique_ptr<GameComponent> cube =
    std::make_unique<RotatingCube>();
cube->SetPosition({2.0f, 1.0f, 0.0f});

const auto added = scene.AddComponent(
    "MyCube", cube, "RotatingCube");
if (added.result != SceneResult::Ok &&
    added.result != SceneResult::Deferred)
    throw std::runtime_error("Cannot spawn cube");
```

Подключите RotatingCube.h и <stdexcept>.

Сцена получает владение объектом.

Имя должно быть уникальным. Во время обновления добавление может быть отложено.

Полный ProjectGame.cpp дополнительно проверяет, что куб ещё не добавлен.

Импортированная модель из C++

```
#include <Engine/Scene/SceneAssets.h>

// Файл уже импортирован через редактор.
auto model = SceneAssets::CreateModel(
    "/Game/Models/blend-packed/blend-packed.glb");
model->SetPosition({-2.0f, 0.0f, 0.0f});

const auto added = game.GetScene().AddComponent(
    "MyModel", model, SceneAssets::ModelType);
if (added.result != SceneResult::Ok &&
    added.result != SceneResult::Deferred)
    throw std::runtime_error("Cannot spawn model");
```

/Game — папка Content проекта.

CreateModel читает .import.json и назначает материал, созданный при импорте FBX/BLEND.

Подставьте путь вашей модели.

Полная функция в ModelExample.h/.cpp: SpawnModel(game, "MyModel", путь_модели).

Материал и текстура из C++

```
// object — GameComponent&, ресурс уже в Content.
const std::string material = "/Game/Materials/Wood.mat";
SceneAssets::ValidateMaterial(material);
if (!object.SetMaterialAsset(material))
    throw std::runtime_error("Material unsupported");

// Отдельная текстура объекта: материал сохраняется.
if (!object.TrySetTexture("/Game/Textures/grid.png"))
    throw std::runtime_error("Cannot set texture");

// Убрать отдельную текстуру, оставив материал:
object.TrySetTexture("");
```

BaseMap меняйте в редакторе материала.

Отдельная текстура объекта дополнительно умножает его цвет.

SetMaterialAsset назначает ссылку; шейдер компилируется при отрисовке.

Для новой геометрии используйте CreateModel. TrySetTexture меняет изображение, а не модель.

Ввод: поля и жизненный цикл

```
#pragma once
// Source/InputCube.h
#include "RotatingCube.h"
#include <Engine/InputDevice/InputActions.h>

ENGINE_COMPONENT("InputCube")
class InputCube : public RotatingCube {
public:
    void BeginPlay() override;
    void Tick(float deltaTime) override;
private:
    bool spinning_ = true;
    unsigned nextCube_ = 1;
    Input::Connection toggle_, spawn_;
    Input::Scope inputScope_;
    void SpawnCube();
};
```

BeginPlay вызывается перед первым игровым вводом и Tick.

Connection нужно хранить.

Scope объявлен последним из полей: он отключит обработчики до разрушения их состояния.

Скопируйте InputCube.h/.cpp из примеров в Source и добавьте один InputCube в сцену после сборки.

Клавиша R: остановить вращение

```
void InputCube::BeginPlay()
{
    RotatingCube::BeginPlay();
    auto& input = GetGame()->GetInput();
    input.Bind({"ToggleRotation",
        Input::Context::Game, {Keys::R}});
    toggle_ = input.Subscribe(inputScope_,
        Input::Context::Game, "ToggleRotation",
        Input::Phase::Pressed,
        [this](const Input::Event&) {
            spinning_ = !spinning_;
        });
    // Здесь же добавьте привязку N со следующего слайда.
}
```

R переключает вращение один раз на нажатие.

В Tick: RotatingCube::Tick(spinning_ ? deltaTime : 0.0f);

После Play щёлкните viewport.

Контекст Game получает ввод в сфокусированном viewport в Play; Pause и потеря фокуса сбрасывают действия.

Клавиша N: создать новый куб

```
// Продолжение InputCube::BeginPlay()
input.Bind({"SpawnCube",
    Input::Context::Game, {Keys::N}});
spawn_ = input.Subscribe(inputScope_,
    Input::Context::Game, "SpawnCube",
    Input::Phase::Pressed,
    [this](const Input::Event&) { SpawnCube(); });

// В InputCube::SpawnCube()
std::unique_ptr<GameComponent> cube =
    std::make_unique<RotatingCube>();
cube->SetPosition(GetLocalPosition() +
    glm::vec3(2.0f * number, 0, 0));
const auto added = scene.AddComponent(
    name, cube, "RotatingCube");
```

SpawnCube получает GetGame()->GetScene().

number — счётчик, name — свободное имя SpawnedCubeN.

В полном файле проверяются имя и результат добавления.

Новый куб появляется в копии Play. После Stop он исчезает вместе с этой копией.

Сочетания и переназначение

```
// В BeginPlay после привязки ToggleRotation
input.Unbind(Input::Context::Game, "ToggleRotation");
input.Bind({"ToggleRotation", Input::Context::Game,
            {Keys::Control, Keys::R}});

// Две привязки одной оси
input.Bind({"MoveX", Input::Context::Game,
            {Keys::D}, 1});
input.Bind({"MoveX", Input::Context::Game,
            {Keys::A}, -1});
```

Обработчик ToggleRotation остаётся прежним.

Ctrl + R — одновременное удержание клавиш.

Для MoveX подпишитесь на Phase::Axis и читайте e.value. A + D даёт ноль.

Своя схема управления

```
class MyController : public CharacterController {
public:
    using CharacterController::CharacterController;
    void BindInput() override {
        game_.GetInput().Bind({"Go",
            Input::Context::Game, {Keys::I}});
        connections_.push_back(game_.GetInput().Subscribe(
            inputScope_, Input::Context::Game, "Go",
            Input::Phase::Axis,
            [this](const Input::Event& e) {
                forward_ = e.value;
            }));
    }
};
```

I задаёт движение вперёд и заменяет стандартные привязки.

Game выбирает контроллер в методе фабрики. Пример — в README.txt.

Для своей камеры переопределите UpdateCamera().

Выберите MyGame: Проект → C++ → Типы C++ проекта…; роль игрока включите в свойствах персонажа.

Разметка и логика игрового UI

Внешний вид	Создайте интерфейс в редакторе UI или измените XML вручную.
Подключение	Настройки проекта выбирают один HUD. Его создаёт движок при запуске игры.
Поведение	C++ обновляет показатели и назначает действия элементам по имени.

Визуальный редактор сохраняет тот же .ui, который читает игра.

Разметка панели HUD

```
<UI>
  <Panel name="stats" rect="16 16 280 144"
         padding="12" color="#101820DD">
    <Label name="clicks" text="Нажатий: 0"
           rect="0 0 240 22"/>
    <Bar name="load" rect="0 32 240 12"
         fill="0" fillColor="#35B8CFFF"/>
    <Button name="add" rect="0 64 240 32"
            text="Добавить"/>
  </Panel>
</UI>
```

Сохраните в Content/UI/Hud.ui, кодировка UTF-8.

name связывает элемент с C++.

padding задаёт отступ для дочерних элементов.

Полный пример: папка UI в архиве исходников туториала.

Размеры и привязки UI

Атрибут	Как работает
rect="x y width height"	Смещение и размер в пикселях внутри родителя
anchor="TopRight"	Отступ от правого верхнего края внутрь панели
anchor="BottomLeft"	Отступ снизу. Для текста задайте высоту явно
width / height = 0	Растянуть до края родительской области
padding="12"	Один отступ или четыре: слева, сверху, справа, снизу
color="#RRGGBBAA"	Цвет и прозрачность. #RRGGBB полностью непрозрачен

Действия выбранного HUD в C++

```
// В классе игры: bool hudBound_ = false;
// unsigned clicks_ = 0;
void ProjectGame::PreUpdate(float dt) {
    auto* hud = GetProjectHud();
    if (!hud) return;
    if (!hudBound_) {
        hud->BindClick("add", [this] { ++clicks_; });
        hudBound_ = true;
    }
    hud->SetLabelText("clicks",
        "Нажатий: " + std::to_string(clicks_));
    hud->SetBarFill("load", float(clicks_ % 11) / 10);
}
```

Выберите .ui через настройки HUD.

Имена элементов: add, clicks, load.

BindClick сохраняется при обновлении.

Подключите UIComponent.h. Для собственного класса игры объявите PreUpdate как override.

Горячее обновление без ручной перепривязки

Назначьте по имени

BindClick хранит действие в компоненте и назначает его новой кнопке после загрузки.

Обновляйте значения

SetLabelText и SetBarFill сами находят текущий элемент. Отсутствующее имя безопасно.

Сохраняйте рабочий UI

При ошибке XML остаётся прежнее дерево. Исправьте файл и сохраните ещё раз.

Find и прямой OnClick остаются низкоуровневым API: их указатели не переживают замену дерева.

Практика с HUD в Battle

Запустите Battle

Запустите Battle с --hud: появятся время кадра, вызовы отрисовки, техника и уровни LOD.

Измените файл

Рядом с Battle.exe откройте GameContent/UI/Hud.ui. Измените color или rect и сохраните.

Проверьте результат

Раскладка обновится. Кнопка и F6 переключают раскраску LOD, F7 скрывает HUD.

Battle включён в дополнительные встроенные примеры. Правки ресурсов делайте в копии проекта.

Если HUD не обновился

Симптом	Что проверить
Нет HUD при запуске	Выбор в «HUD игры», файл .ui и сообщение в логе
Старый вид после сохранения	Загруженную копию .ui и строку ошибки XML
Изменение атрибута не видно	Неизвестный атрибут игнорируется с предупреждением
Кнопка без действия	Уникальное name и BindClick в коде игры
Камера реагирует на щелчок	interactive и использование игрового Input API
Символы пропали	Атлас содержит ASCII и русские буквы А-я, Ё и ё

Обновление движка и проекта

Обычный запуск

Нажмите «Играть» или «Редактировать» у старого проекта.

Предложение

Лаунчер покажет версии и предложит обновить копию проекта.

Выбор

Можно открыть исходник на прежней установленной версии или отменить.

Обновление сохраняет исходную папку. Ручной способ: «Обновить копию проекта…».

Частые вопросы при первом запуске

Симптом	Что проверить
Персонаж не двигается	Play, фокус viewport, назначение и флажок управления
BLEND не импортируется	Blender 4.2+, доступные картинки или упакованные текстуры
Материал рисуется шахматкой	Ошибки HLSL, путь .mat; после исправления нажмите F6
Новый класс не виден	Успешная сборка, регистрация и Hot Reload в Edit
Ошибка сборки / загрузки	Консоль, CodeBuild.log и логи кандидата в Binaries
Пропала модель или текстура	Ресурс импортирован в Content и перенесён вместе с проектом

Редактор: схемы и раскладка

Управление	Две схемы: Unreal / Unity и Blender. Выбор находится в меню «Правка».
Окна	Перенос и сохранение панелей, темы, язык и сброс раскладки через «Вид».
Viewport	Gizmo, сетка и FXAA получили исправления выбора, глубины и очистки кадра.

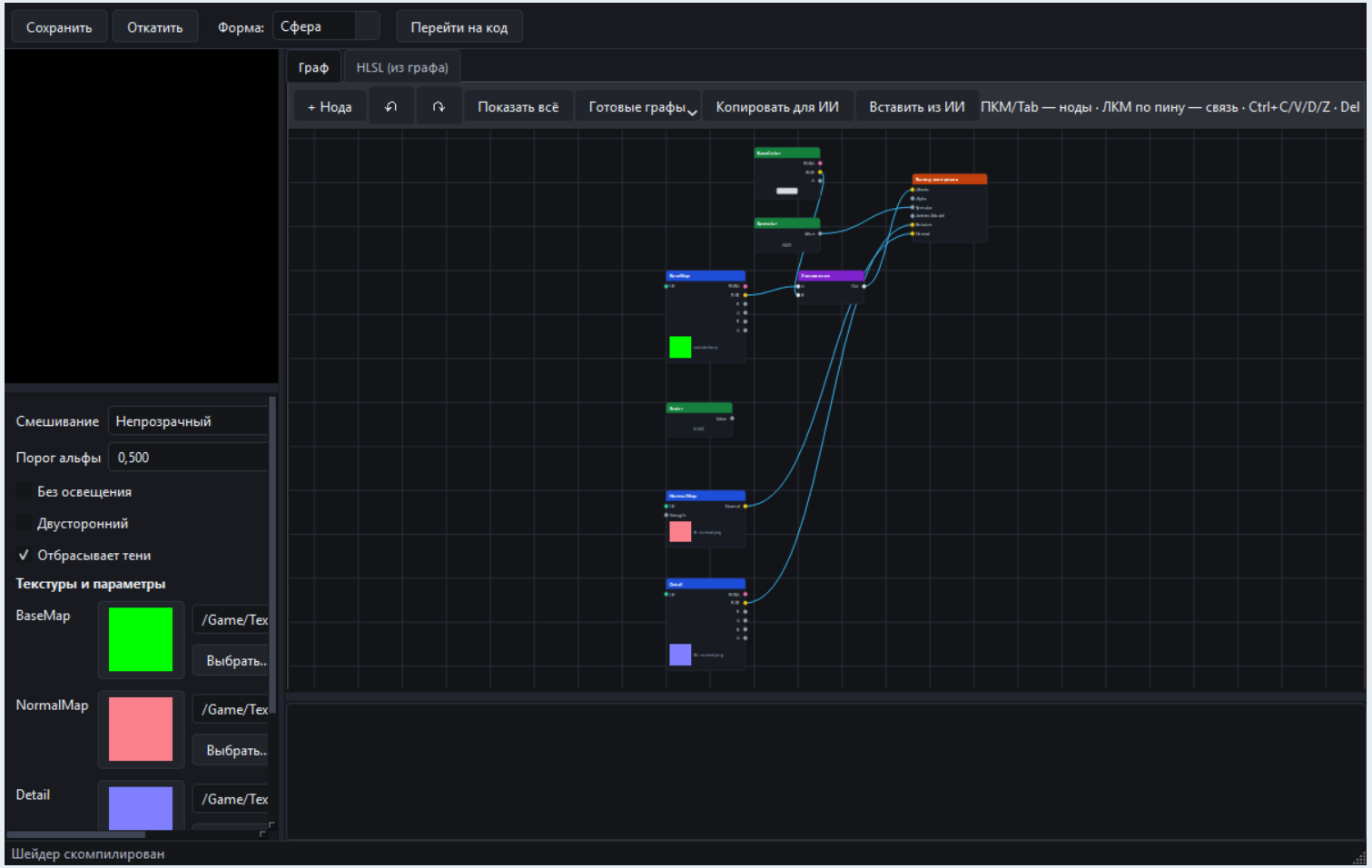
Материалы: узловой граф

Ноды и связи

Tab открывает поиск. Соедините выход ноды со входом Surface.

Результат

Граф создаёт HLSL. Сохраните .mat и .mat.graph.json вместе.



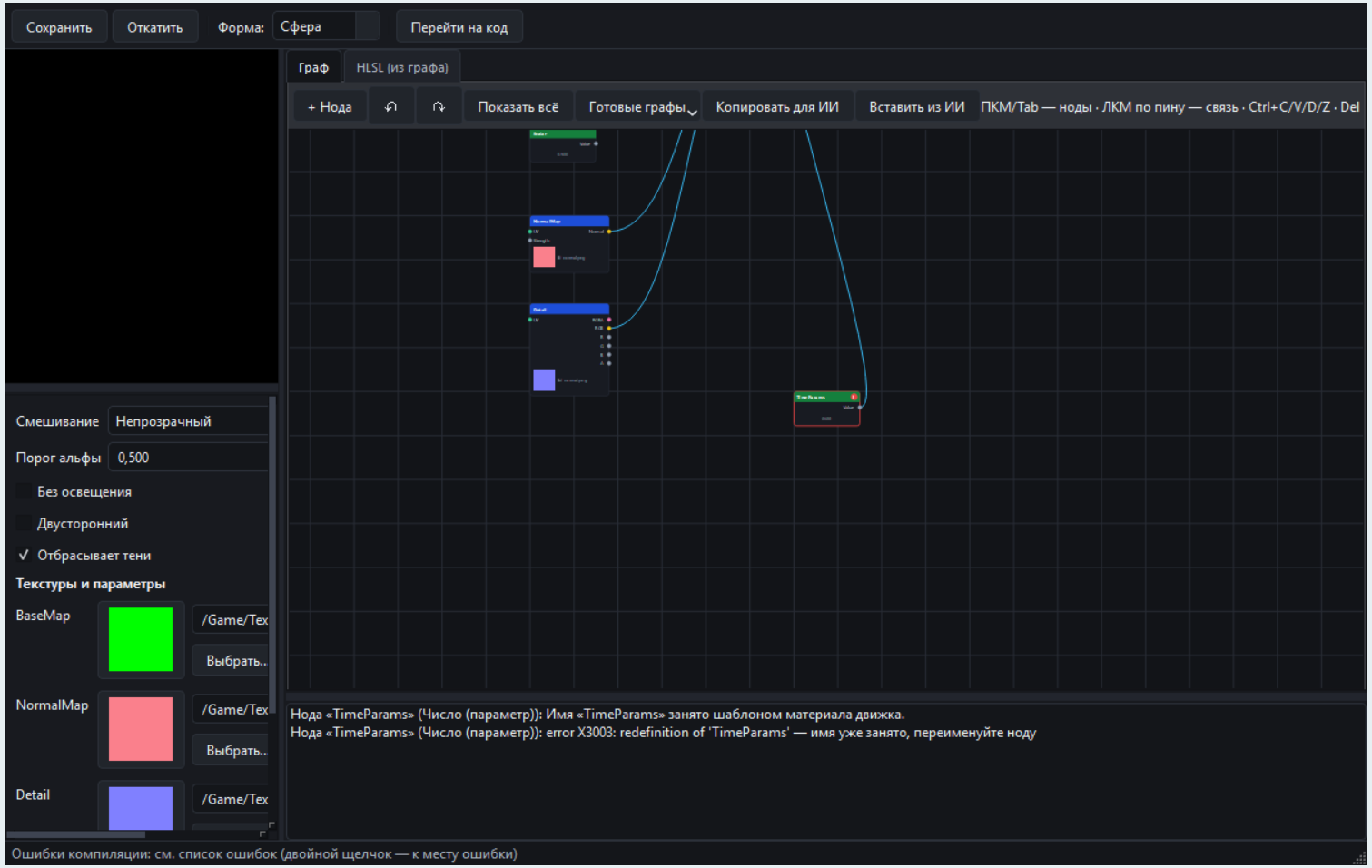
Граф: поиск, отмена и ошибки

Правки

Ctrl+Z / Ctrl+Y, копирование, вставка и дублирование нод.

Диагностика

Проверьте ошибку графа. Откат восстанавливает согласованные файлы.

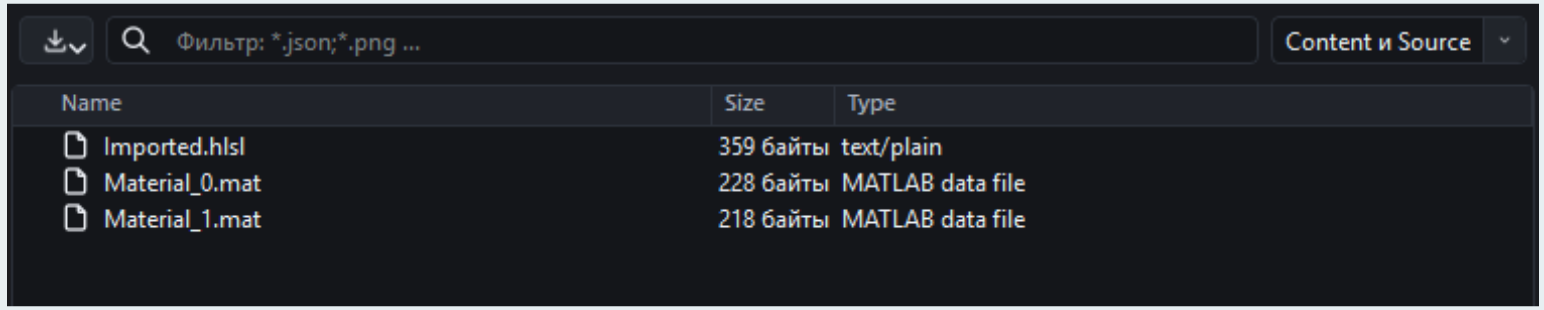


Материалы модели: где редактировать

1. Файлы слотов

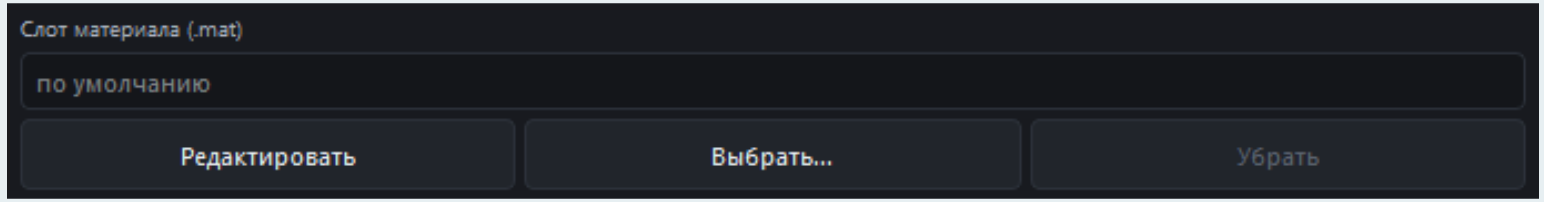
В Content/Models откройте папку модели, затем Materials. Двойной щелчок по Material_0.mat открывает редактор.

Content/Models/blend-suzanne-multiple/Materials



2. Общий материал

«Убрать» снимает общий override. Исходные назначения слотов сохраняются.



Пример: CuriousCity/Content/Models/blend-suzanne-multiple/Materials

Материалы модели: замена слота в C++

CuriousCity / Source / ProjectGame.cpp (переносы для слайда)

```
const auto slots = skeletal->GetMaterialSlots();
if (slots.empty()) continue;
const uint32_t sourceId = slots.begin()->first;
const std::string material =
    "/Game/Models/CesiumMan/Materials/Material_0.mat";
const bool slotSet =
    skeletal->SetMaterialSlot(sourceId, material);
// ... проверка общего override ...
const bool overrideCleared =
    skeletal->SetMaterialAsset("");
```

Фрагмент цикла из CuriousCity.

ID берём из GetMaterialSlots(): номера могут иметь пропуски.

Проверяйте bool. Очистка общего материала возвращает отдельные слоты.

Полный пример: Examples/CuriousCity/Source/ProjectGame.cpp

Звук в редакторе: tutorial.wav

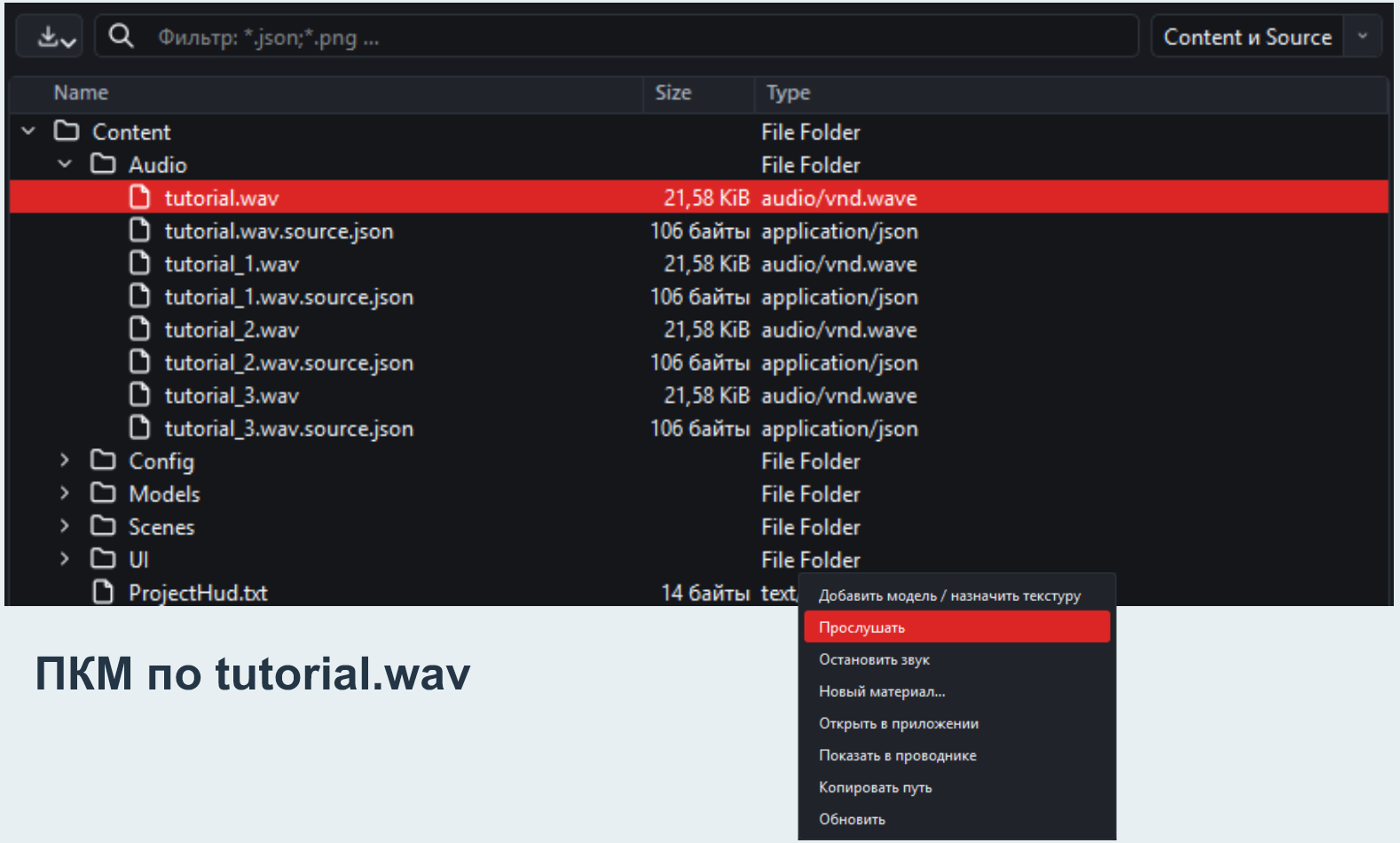
1. Импорт

Сцена → Ресурсы → Импорт аудио… Выберите tutorial.wav.

2. Прослушивание

ПКМ по файлу в Content/Audio → «Прослушать».

«Остановить звук» завершает предпрослушивание.



ПКМ по tutorial.wav

Учебный файл: Tutorials/LearningLab/Content/Audio/tutorial.wav

Звук из игрового C++

```
auto* audio = GetGame()->GetAudioEngine();
if (!audio) return;
auto path = EngineModuleVfs::Resolve(
    "/Game/Audio/tutorial.wav");
auto id = audio->LoadSound(
    std::filesystem::u8path(path).wstring());
if (!id.IsValid()) return;
audio::AudioEngine::PlayOptions options;
options.volume = 0.25f;
audio->Play2D(id, options);
```

Фрагмент BeginPlay.

Полный класс и includes: AudioTutorial.h/.cpp

Game управляет временем жизни AudioEngine.

3D-звук и потоковая музыка

Короткий эффект	LoadSound и Play3D используют SoundId и копию AudioEmitter3D.
Музыка	PlayStream читает файл потоково. Сохраните StreamHandle для StopStream.
Ограничение	Для активного 3D-источника пока нет публичного handle перемещения.

Игровые подсистемы: один счётчик на сессию

SessionCounterSubsystem.h (переносы для слайда)

```
#pragma once
#include <Engine/MainGame/GameSubsystem.h>

ENGINE_SUBSYSTEM()
class SessionCounterSubsystem final : public GameSubsystem
{
public:
    SessionCounterSubsystem() { bTickEnabled = true; }
    unsigned scenesOpened = 0;
    unsigned ticks = 0;
    void OnSceneReady(Scene&) override { ++scenesOpened; }
    void Tick(float) override { ++ticks; }
};
```

Добавьте .h в Source проекта и выполните Build.cmd. Генератор найдёт ENGINE_SUBSYSTEM() .

OnSceneReady считает сцены. Tick работает, потому что bTickEnabled = true.

Реальный исходник: Game/BasicExamples/Source/SessionCounterSubsystem.h

Подсистема: получение и срок жизни

```
#include "SessionCounterSubsystem.h"
#include <Engine/MainGame/BaseGameClass/Game.h>

// game — текущий Game& на игровом потоке
auto* counter =
    game.GetSubsystem<SessionCounterSubsystem>();
if (counter) {
    const auto opened = counter->scenesOpened;
    const auto ticks = counter->ticks;
    // Передайте opened и ticks в свой HUD.
}
```

Смена сцены сохраняет экземпляр.

OnSceneClosing очищает ссылки на старую сцену, OnSceneReady привязывает новые.

Stop уничтожает подсистему. Hot Reload не переносит её состояние.

Для экономики готовый пример: Examples/CuriousCity/Source/CityEconomySubsystem.*

SaveGame: данные экономики

CuriousCity / Source / CityEconomySubsystem.cpp (переносы для слайда)

```
void CityEconomySubsystem::SerializeSaveGame(
    SaveArchive& archive)
{
    GameSubsystem::SerializeSaveGame(archive);
    archive.Section("CuriousCity.Economy",
        [&](SaveArchive& state) {
        state.Version(1);
        state.Field("balance", balance);
        state.Field("houses", houses);
        state.Field("incomeClock", incomeClock);
    });
}
```

Вызов базы сохраняет контракт подсистемы.

Section отделяет данные экономики. Version задаёт версию секции. Field работает при записи и чтении.

Сохраняются баланс, число домов и таймер дохода.

Examples/CuriousCity/Source/CityEconomySubsystem.cpp · подключение: Engine/SaveGame/SaveArchive.h

SaveGame: запрос и завершение

```
#include <Engine/SaveGame/SaveGame.h>
#include <Engine/MainGame/BaseGameClass/Game.h>

// При действии «Сохранить»:
const auto request = game.Saves().RequestSave("city");
const auto ticket = request.ticket;

// В последующих кадрах, на том же игровом потоке:
const auto& result = game.Saves().LastResult();
if (result.ticket == ticket &&
    result.status == SaveGameResult::Status::Saved) {
    // Теперь можно показать «Сохранено».
}
// Загрузка: game.Saves().RequestLoad("city");
```

Accepted означает постановку в очередь.

Сохраните ticket между кадрами. Обрабатывайте Failed и message, а также отказ Busy.

После Loaded прежние указатели на сцену недействительны.

В CuriousCity кнопки Save / Load связаны с RequestSave / RequestLoad.

Код: создание, сборка и запуск

```
EngineLauncher.exe --create-project "MyDemo" "MyDemo"
EngineLauncher.exe --enable-code "MyDemo/Project.json"

rem После изменения Source/*.cpp:
EngineLauncher.exe --build-project "MyDemo/Project.json"

Runtime/EngineRuntime.exe --project "MyDemo/Project.json" ^
    --mode editor

rem Готовый учебный проект:
Tutorials/LearningLab/Build.cmd
Tutorials/LearningLab/Run.cmd
```

Команды для cmd.exe из корня комплекта.

Нужны MSVC v143 и Windows SDK.

При ошибке Hot Reload рабочая DLL остаётся активной. Исправьте ошибку компилятора и повторите сборку.

Для существующего проекта пропустите create-project и enable-code.

Диагностика: данные завершённого кадра

CuriousCity / Source / ProjectGame.cpp (переносы для слайда)

```
const auto& frame = GetCompletedFrameStats();
if (frame.valid && frame.frameIndex != previous) {
    previous = frame.frameIndex;
    log << "completed_frame valid=true index="
        << frame.frameIndex
        << " draw_calls=" << frame.counters.DrawCalls
        << " visible=" << frame.counters.VisibleObjects
        << " culled=" << frame.counters.CulledObjects
        << '\n';
}
```

ProjectGame::PreUpdate читает предыдущий завершённый кадр.

F7 показывает панель. F8 выбирает InstancedMeshComponent.

DrawCalls включает тени и другие проходы. Это не GPU timing и не FPS.

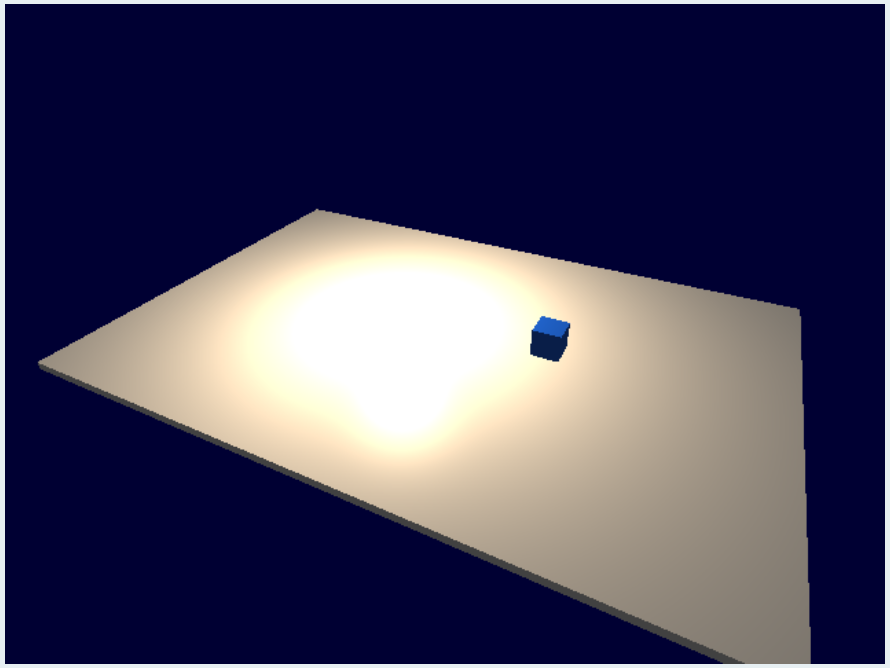
Examples/CuriousCity/Source/ProjectGame.cpp · полный контекст: PreUpdate

Исправления теней и нормалей

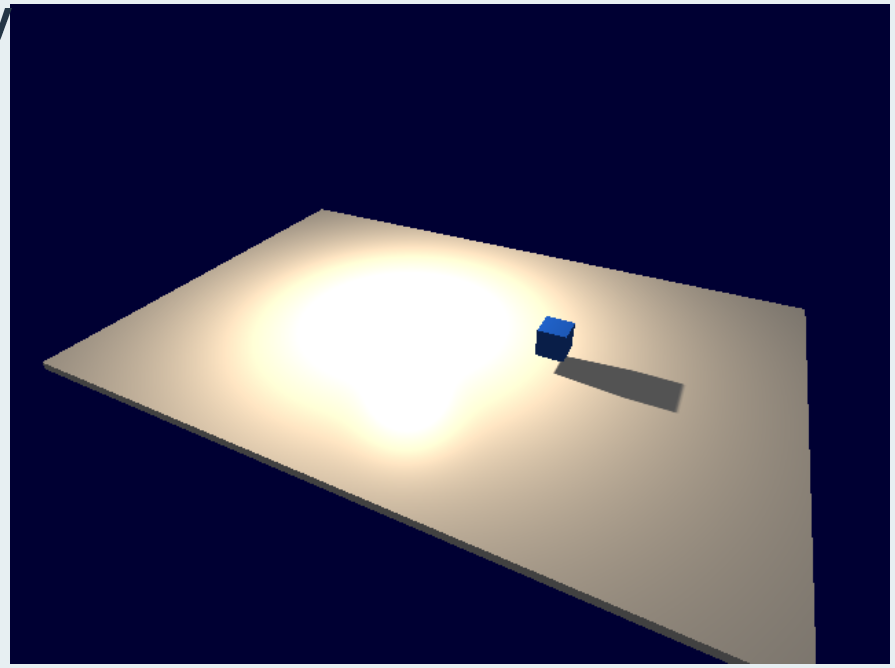
Каскады	Исправлены выбор и устойчивость солнечных каскадов при изменении камеры.
Точечный свет	Тени следуют за лампой и анимацией. Следующие слайды показывают настройку и границы.
Normal map	GPU получает TANGENT. Зеркальный масштаб корректирует знак базиса.

Точечный свет: тень движущегося объекта

До: VAT-поза выходит за bind sphere



После: тень учитывает анимированную позу



Старая граница отсечения пропускала объект. Теперь sampled poses задают общий envelope. Карты обновляются при движении лампы и объекта.

Регрессия: 34/34 сценария движения, 282 кадра. Повторена в дистрибутиве .6.

Точечный свет: настройка и ограничения

```
#include <Engine/Components/Light/PointLightComponent.h>
#include <Engine/Scene/Scene.h>
#include <memory>

// scene — текущая Scene&
auto lamp = std::make_unique<PointLightComponent>(
    glm::vec3(0, 4, 0), glm::vec3(1, 0.7f, 0.4f),
    1.4f, 12.0f); // интенсивность, радиус
lamp->SetCastShadows(true);
std::unique_ptr<GameComponent> object = std::move(lamp);
scene.AddComponent("Lamp", object, "Engine.PointLight");
```

До двух теневых ламп одновременно.

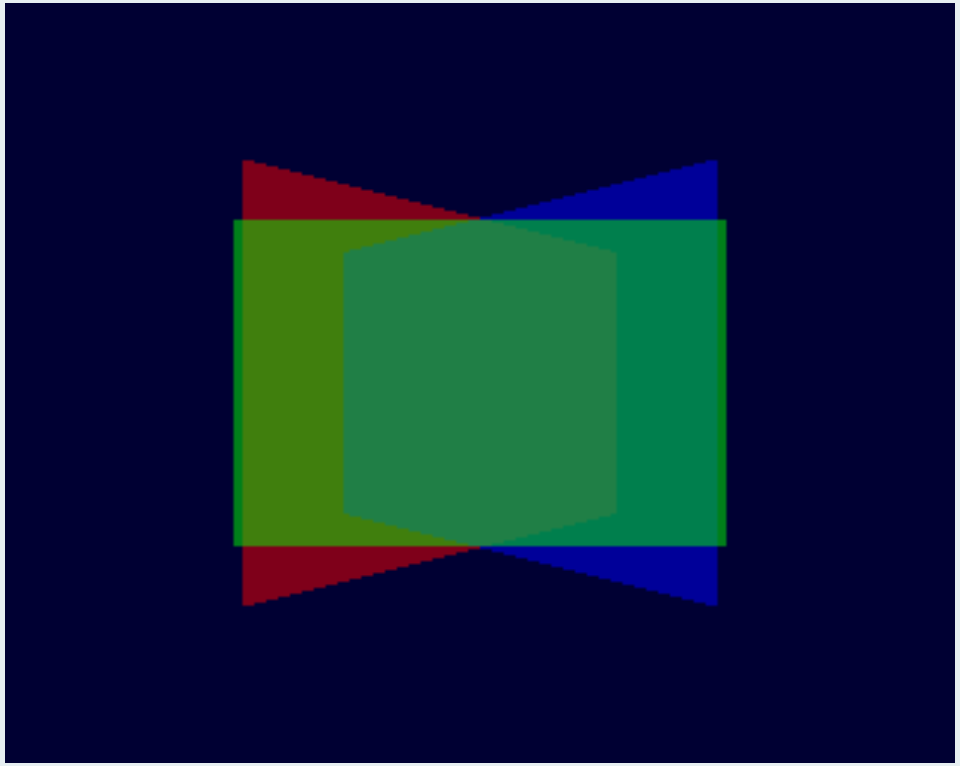
Куб 1024 × 1024 × 6 на лампу. Резерв двух кубов: 48 MiB.

Intensity = 0 освобождает слот. Mesh LOD-тень следует dither-переходу.

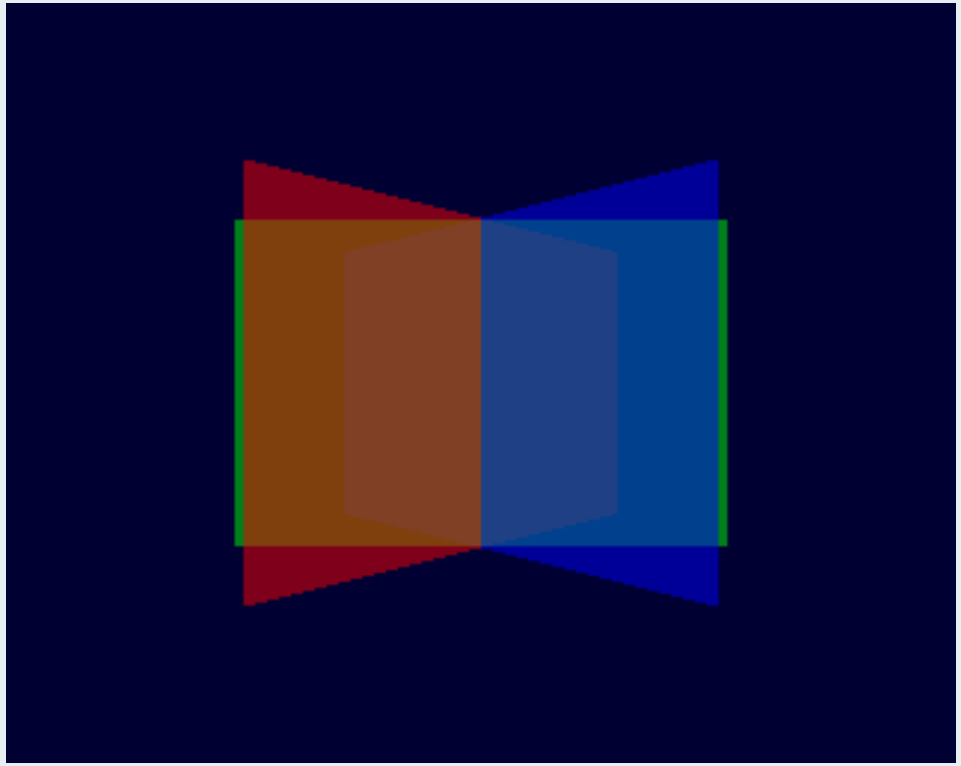
Distance LOD без fade переключается резко. Для impostor тень сохраняет последний mesh.

Прозрачность: пересечение поверхностей

До: один порядок на объект



Теперь: порядок по пикселю



Depth peeling обрабатывает 16 ближайших различных глубин. LOD-dither больше не смешивает один и тот же слой дважды.

Реальные кадры до и после исправления; одинаковая тестовая геометрия.

Прозрачность: измеренная точность

Проверка	Результат
Исходная матрица до фикса	0/20 сценариев · 280 кадров
Та же матрица после фикса	20/20 · 280 кадров
Расширенный набор выпуска .6	46/46 · 556 кадров
Проверено пиксельных пересечений	297 230 · максимум ошибки 1/255
Регрессия слоёв и LOD	52/52 · 428 кадров

Windows x64 Release · RTX 4070 Laptop · D3D11 Forward / Deferred · сохранённый прогон 07.11

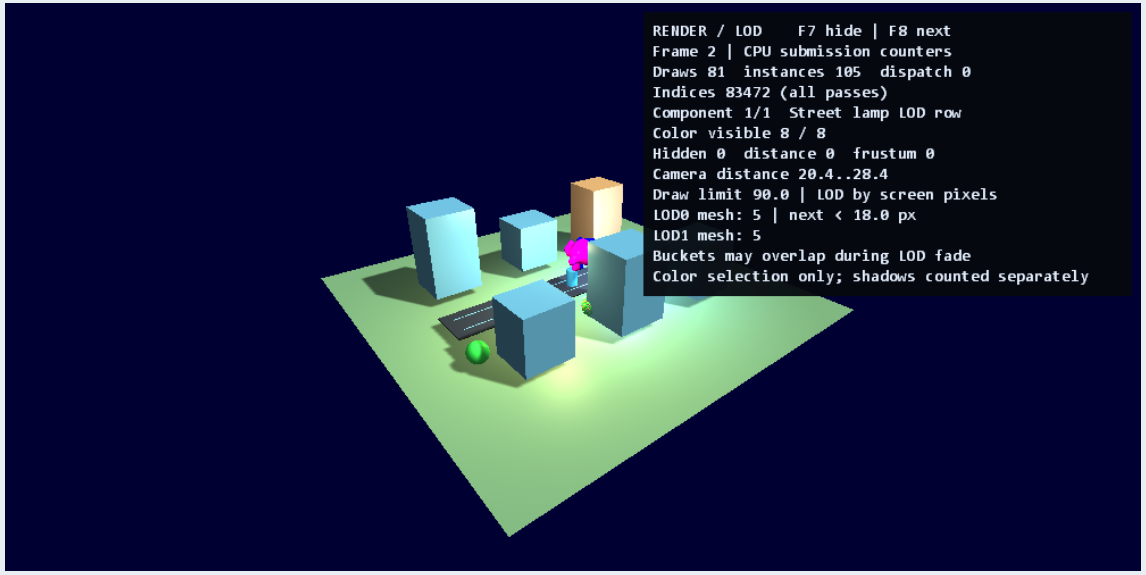
Прозрачность: стоимость и границы

Метрика	Значение
Экранные текстуры	24 байта/пиксель ≈ 47,5 MiB при 1920×1080
Геометрические проходы	До 33
Композиция	16 промежуточных + 1 финальная
Контрольная сцена, mesh normal	116 отправленных draw calls / кадр
После 16 глубин или legacy PS	Запасная сортировка с ограничениями

Draw calls включают команды, пропущенные GPU predicate. FPS и GPU-время здесь не измерялись.

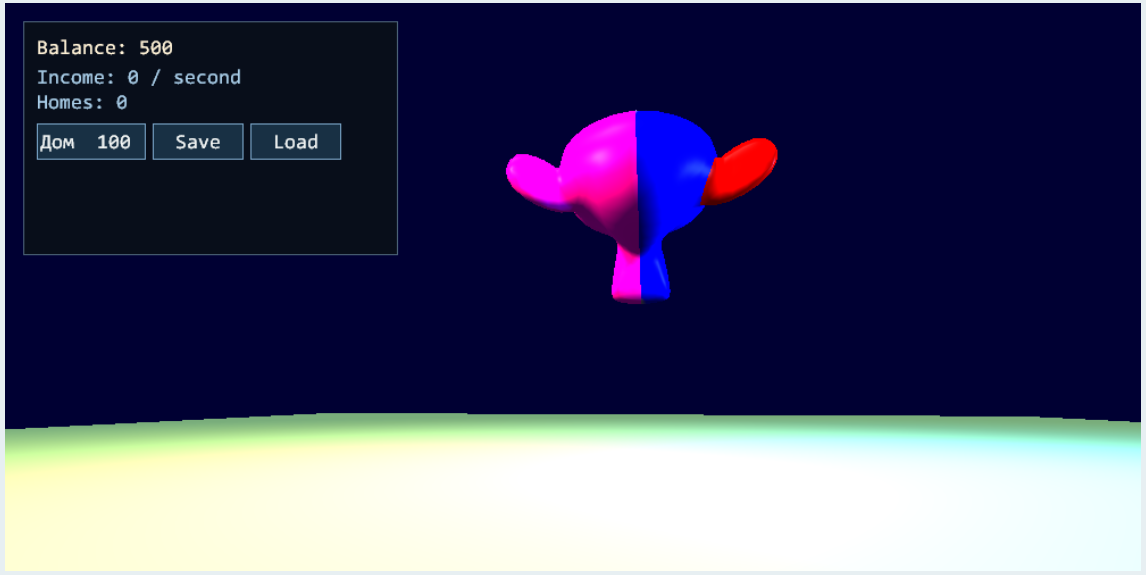
Примеры: движение, экономика и SaveGame

CalmCity



Движущаяся машина, квартал и LOD.
Смотрите Source/ProjectGame.cpp:
состояние города и сохранение.

CuriousCity

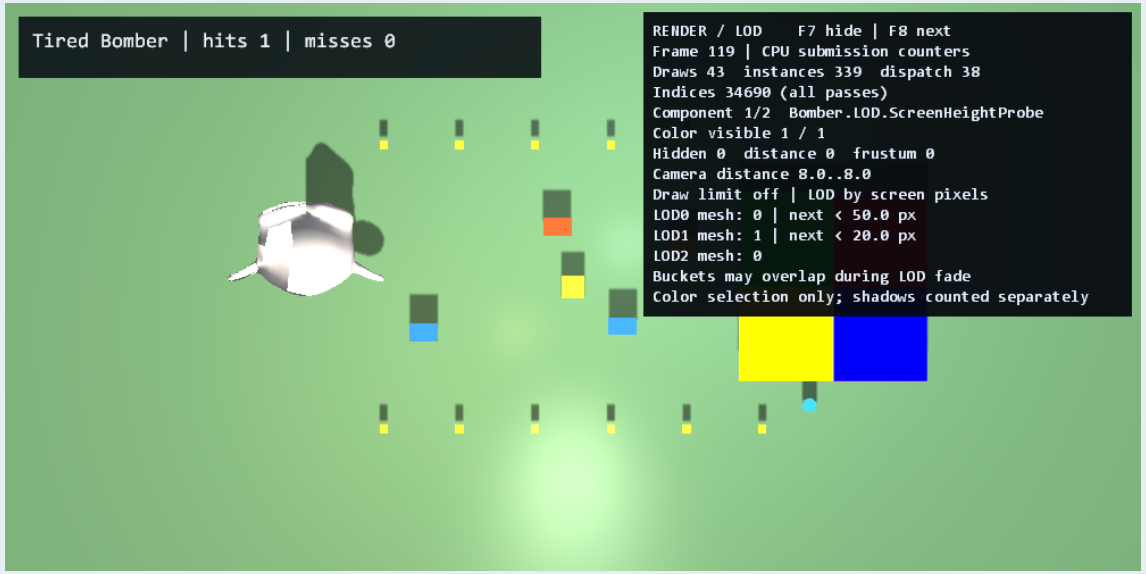


«Дом 100» строит дом и уменьшает баланс.
Save / Load сохраняют город.
Source/CityEconomySubsystem.cpp

Examples/<имя>/Run.cmd — запуск · Open.cmd — редактор · Source/ — код · Content/ — ресурсы

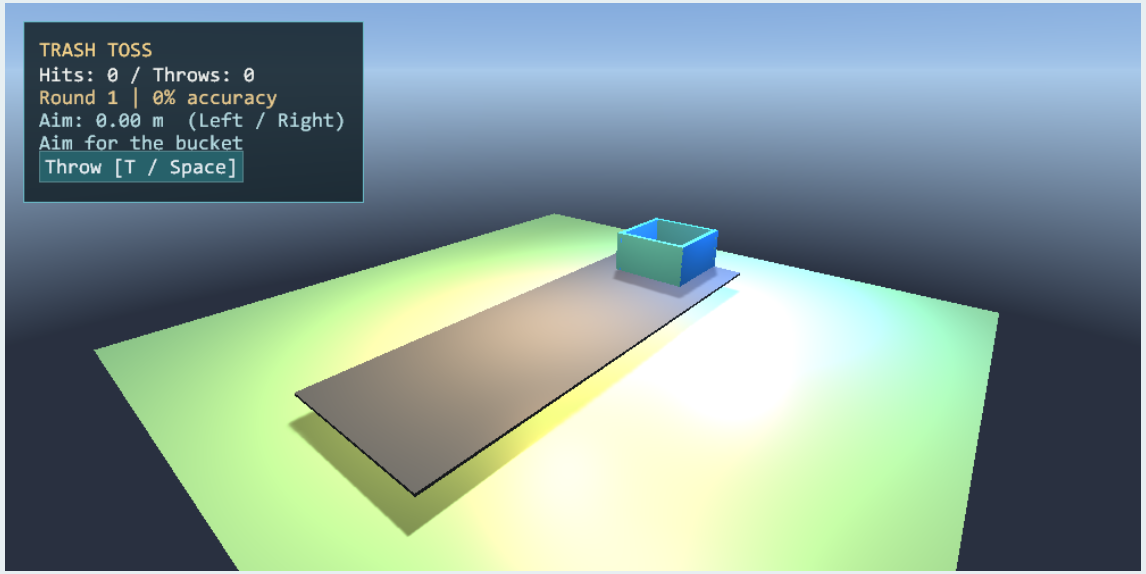
Примеры: действие, попадание и счёт

TiredBomber



В сбрасывает бомбу. На кадре Hits: 1.
Разберите трассировку попадания, эффекты и сохранение раунда.

AITrash



Left / Right меняют прицел. T или Space бросает предмет.
HUD показывает броски и попадания.

Examples/<имя>/Run.cmd — запуск · Open.cmd — редактор · Source/ — код · Content/ — ресурсы

Учебный комплект: точное расположение

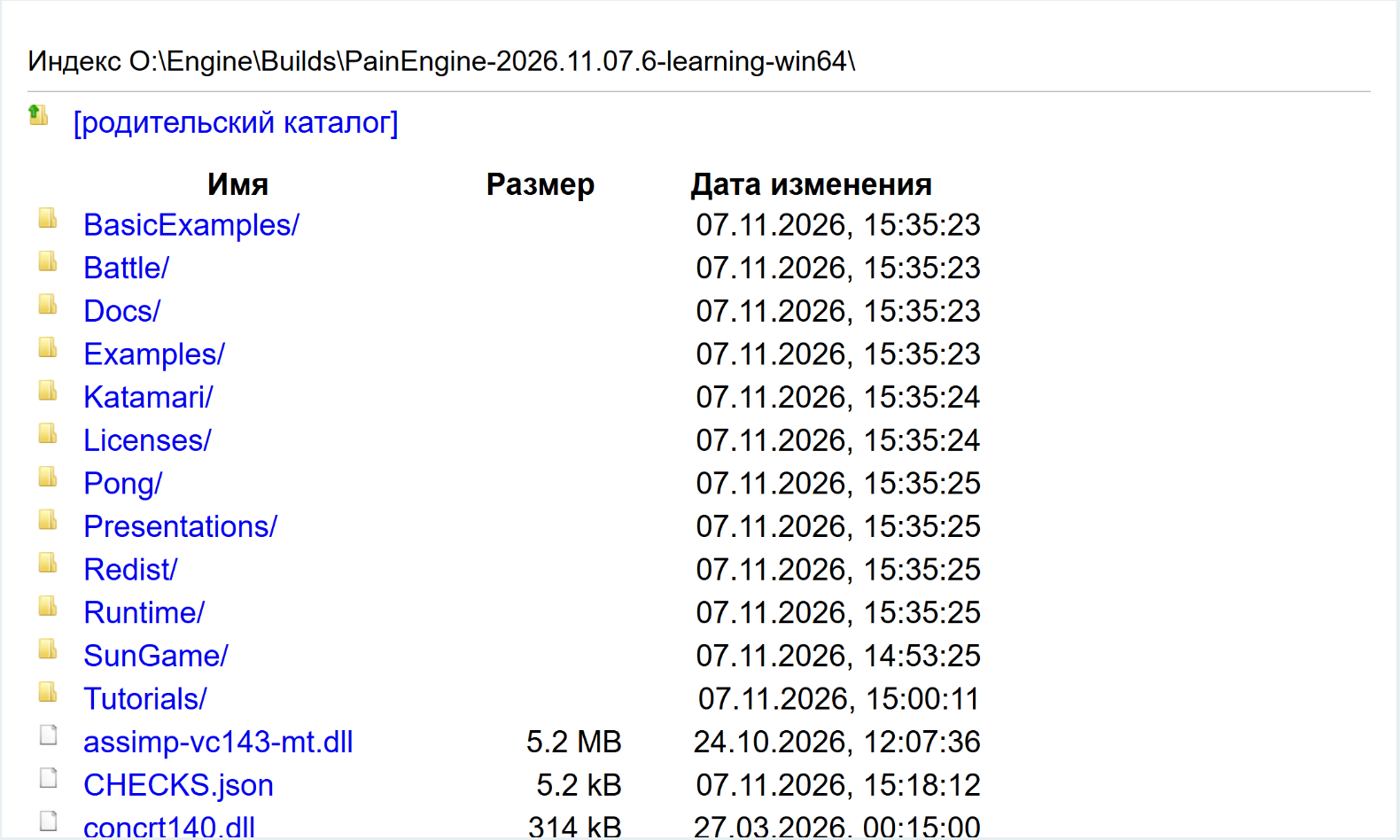
В корне распакованного архива

README.html
Порядок первого запуска

Docs/index.html
Руководства и API

Tutorials/LearningLab
Build.cmd и Run.cmd

Examples/<имя>
Четыре проекта с кодом


Индекс O:\Engine\Builds\PainEngine-2026.11.07.6-learning-win64\

[родительский каталог]

Имя	Размер	Дата изменения
BasicExamples/		07.11.2026, 15:35:23
Battle/		07.11.2026, 15:35:23
Docs/		07.11.2026, 15:35:23
Examples/		07.11.2026, 15:35:23
Katamari/		07.11.2026, 15:35:24
Licenses/		07.11.2026, 15:35:24
Pong/		07.11.2026, 15:35:25
Presentations/		07.11.2026, 15:35:25
Redist/		07.11.2026, 15:35:25
Runtime/		07.11.2026, 15:35:25
SunGame/		07.11.2026, 14:53:25
Tutorials/		07.11.2026, 15:00:11
assimp-vc143-mt.dll	5.2 MB	24.10.2026, 12:07:36
CHECKS.json	5.2 kB	07.11.2026, 15:18:12
concrt140.dll	314 kB	27.03.2026, 00:15:00

Путь на снимке — пример распаковки. Внутри комплекта используются относительные пути.

Учебный комплект: полный исходник звука

Tutorials / LearningLab

Source/AudioTutorial.h
Объявление компонента

Source/AudioTutorial.cpp
Загрузка и Play2D

Content/Audio/tutorial.wav
Готовый PCM WAV

Build.cmd, затем Run.cmd
Сборка и проверка

Индекс O:\Engine\Builds\PainEngine-2026.11.07.6-learning-win64\Tutorials\LearningLab\Source\

[родительский каталог]

Имя	Размер	Дата изменения
AudioTutorial.cpp	923 B	07.11.2026, 14:59:08
AudioTutorial.h	252 B	07.11.2026, 14:59:08
InputCube.cpp	1.5 kB	07.11.2026, 14:59:08
InputCube.h	409 B	07.11.2026, 14:59:08
ModelExample.cpp	1.1 kB	07.11.2026, 14:59:08
ModelExample.h	373 B	07.11.2026, 14:59:08
Module.cpp	493 B	07.11.2026, 14:59:08
MyController.h	622 B	07.11.2026, 14:59:08
ProjectComponent.cpp	178 B	07.11.2026, 14:59:08
ProjectComponent.h	313 B	07.11.2026, 14:59:08
ProjectGame.cpp	1.7 kB	07.11.2026, 14:59:08
ProjectGame.h	332 B	07.11.2026, 14:59:08
ReferenceComponent.h	1.1 kB	07.11.2026, 14:59:08
ReferenceExamples.cpp	4.8 kB	07.11.2026, 14:59:08
RotatingCube.cpp	418 B	07.11.2026, 14:59:08

Для сборки: Visual Studio C++ Build Tools v143 и Windows SDK.